claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde
xer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-
a08b1f55794109e19.jsonl`
- SHA-256: `ba3263bb1278e32418ec3bc8f7b25d59288bb324e858d2673dc5f01e05f72720`
- Source modified: `2026-07-02T02:55:34+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T02:47:11.980Z)

```
CONTEXT (read carefully):
- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-
repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos
were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra
(SPA web/ + marketing site/, TS/React, org avidullu), sports-data-collector, rally-annotator,
sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-
screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees,
do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-
only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-
only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a
hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify
anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner_gated=true, still report): alpha/serving go-live + Cloudflare
dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing
decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR
#390 (D1 split main.py, open since 06-26) · audit doc docs/CODE_CLEANUP_AUDIT_2026-06-24.md
remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/O3/O4) ·
docs/DECISION_SNAPSHOT_REGRESSION.md next=P1 · docs/GOLDEN_REGRESSION_FIXTURES.md backlog
M4/P0-rename/O1/O2 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound
decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet
RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was
pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security,
tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value
debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have
(report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked_in when an
issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data
for an orchestrator, not prose for a human.

YOUR DIMENSION: code health of the khelsutra repo (SPA + marketing site). Repo:
C:/Users/avidu/Projects/khelsutra-guru/khelsutra.
Check: web/ (React SPA) - TS strictness, test coverage state (vitest), component size
(VideoLibrary.tsx just landed in #140), API-client drift vs the engine's current endpoints
(compare web/src API calls to C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/main.py + backend/api/ routes - any endpoint the SPA calls that changed shape?);
the ENGINE BUNDLE-SYNC convention: the engine vendors a built SPA at
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/ui/ +
```

backend/api/static/app.js - is that bundle STALE vs khelsutra web/ current source (compare build hashes/dates/marker strings)? site/ (marketing) - i18n key parity across the 6 locales (a key-parity check script may exist), dead deploy configs referencing the RETIRED droplet (deploy/, .github/workflows - the parity monitor was removed in #117; check leftovers like cloudflared configs, DEPLOY_ALPHA.md accuracy, GPU_VM_SERVING.md). node_modules junction gotchas aside, do NOT run npm install.

2. user (2026-07-02T02:47:18.860Z)

```
f9eea14 web: resilient video-summary readers + show fps in the library (#140)
9223ba9 docs: refresh khelsutra docs for today's storage-track work (P2-P5) (#139)
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
a04a21b docs(tracker): inline the D14 decisions into the P2-P5 rows (#136)
NEXT_STEPS.md
README.md
deploy
docs
locales
node_modules
site
web
```

3. user (2026-07-02T02:47:19.496Z)

```
site:
README.md
access.db
make_og.py
node_modules
package-lock.json
package.json
public
schema.sql
scripts
server.js
src
wrangler.toml

web:
README.md
dist
e2e
index.html
node_modules
package-lock.json
package.json
src
tsconfig.json
vite.config.ts

web/src:
App.tsx
annotator
api
components
hooks
i18n
index.css
lib
main.tsx
test
vite-env.d.ts
```

4. user (2026-07-02T02:47:27.600Z)

```
{
  "compilerOptions": {
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,

    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",

    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,

    "types": ["vite/client", "node"]
  },
  "include": ["src", "vite.config.ts"]
}

```

5. user (2026-07-02T02:47:28.059Z)

```
{
  "name": "khelsutra-web",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "description": "KhelSutra Studio ? operator console SPA (drives the rally indexer engine).",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "typecheck": "tsc --noEmit",
    "test": "vitest run",
    "test:coverage": "vitest run --coverage",
    "e2e": "playwright test -c e2e/playwright.config.mjs",
    "screencast": "vite build && playwright test -c e2e/playwright.config.mjs",
    "screencast:mp4": "node e2e/to-mp4.mjs"
  },
  "dependencies": {
    "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
    "@fontsource-variable/noto-sans-kannada": "^5.2.9",
    "i18next": "^26.3.1",
    "i18next-browser-languagedetector": "^8.2.1",
    "i18next-icu": "^2.4.3",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "react-i18next": "^17.0.8"
  },
  "devDependencies": {
    "@playwright/test": "^1.56.0",
    "@tailwindcss/vite": "^4.0.0",
    "@testing-library/dom": "^10.4.1",
    "@testing-library/react": "^16.3.2",
    "@types/node": "^26.0.0",
```

```
    "@types/react": "^19.0.0",
    "@types/react-dom": "^19.0.0",
    "@vitejs/plugin-react": "^4.3.4",
    "@vitest/coverage-v8": "^3.2.6",
    "jsdom": "^29.1.1",
    "tailwindcss": "^4.0.0",
    "typescript": "^5.7.0",
    "vite": "^6.0.0",
    "vitest": "^3.0.0"
  }
}

```

6. user (2026-07-02T02:47:29.848Z)

```
src/annotator:
VENDOR.md
annotator.smoke.test.ts
annotator.ts
csv.ts

src/api:
client.test.ts
client.ts
locale.test.ts
types.ts
upload.test.ts

src/components:
AnnotatePanel.test.tsx
AnnotatePanel.tsx
CompileResult.tsx
FfmpegBanner.test.tsx
FfmpegBanner.tsx
IngestPanel.test.tsx
IngestPanel.tsx
LanguageSwitcher.tsx
QueryBar.examples.test.ts
QueryBar.test.tsx
QueryBar.tsx
RallyGrid.tsx
SelectionFooter.test.tsx
SelectionFooter.tsx
SetupWizard.test.tsx
SetupWizard.tsx
StatusChip.tsx
TapFilters.test.tsx
TapFilters.tsx
VideoLibrary.test.tsx
VideoLibrary.tsx
a4.touch.test.tsx
a8.status.test.tsx
build-error.test.tsx
upload.test.tsx

src/hooks:
useIngestJob.test.ts
useIngestJob.ts
useRallySelection.ts
useUpload.test.ts
useUpload.ts

src/lib:
errorText.test.ts
errorText.ts
```

```
errors.test.ts
errors.ts
floor.test.ts
floor.ts
format.test.ts
format.ts
log.ts
query.test.ts
query.ts
selection.test.ts
selection.ts
video.test.ts
video.ts

src/test:
setup.ts
src/annotator/annotator.smoke.test.ts
src/api/client.test.ts
src/api/locale.test.ts
src/api/upload.test.ts
src/components/a4.touch.test.tsx
src/components/a8.status.test.tsx
src/components/AnnotatePanel.test.tsx
src/components/build-error.test.tsx
src/components/FfmpegBanner.test.tsx
src/components/IngestPanel.test.tsx
src/components/QueryBar.examples.test.ts
src/components/QueryBar.test.tsx
src/components/SelectionFooter.test.tsx
src/components/SetupWizard.test.tsx
src/components/TapFilters.test.tsx
src/components/upload.test.tsx
src/components/VideoLibrary.test.tsx
src/hooks/useIngestJob.test.ts
src/hooks/useUpload.test.ts
src/i18n/catalog-invariants.test.ts
src/i18n/hygiene.test.ts
src/i18n/i18n.test.tsx
src/lib/errors.test.ts
src/lib/errorText.test.ts
src/lib/floor.test.ts
src/lib/format.test.ts
src/lib/query.test.ts
src/lib/selection.test.ts
src/lib/video.test.ts
```

7. user (2026-07-02T02:47:39.172Z)

```
  4611 total
   417 src/App.tsx
   376 src/components/IngestPanel.tsx
   341 src/api/client.ts
   312 src/components/IngestPanel.test.tsx
   233 src/api/client.test.ts
   229 src/components/AnnotatePanel.tsx
   186 src/api/types.ts
   179 src/components/SetupWizard.tsx
   173 src/hooks/useIngestJob.test.ts
   147 src/hooks/useIngestJob.ts
   127 src/components/SetupWizard.test.tsx
   124 src/components/build-error.test.tsx
   120 src/components/SelectionFooter.test.tsx
   114 src/components/QueryBar.tsx
   110 src/components/a8.status.test.tsx
   101 src/components/VideoLibrary.tsx
```

```
    96 src/components/CompileResult.tsx
    93 src/components/upload.test.tsx
    93 src/api/upload.test.ts
    88 src/hooks/useUpload.ts
    87 src/components/a4.touch.test.tsx
    81 src/components/SelectionFooter.tsx
    79 src/components/RallyGrid.tsx
    75 src/hooks/useUpload.test.ts
```

8. user (2026-07-02T02:47:39.413Z)

```
// Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the single-origin
// appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ? :8000).
// Every call carries a correlation id (header + logs) so a journey is traceable end to end.
import { log, newCorrelationId } from "../lib/log";
import type {
  RallySegment,
  QueryResponse,
  CompileResult,
  EngineConfig,
  EngineCapabilities,
  ProcessResponse,
  JobStatus,
  UploadInitResponse,
  UploadPartResponse,
  UploadCompleteResponse,
  VideoSummary,
  VideosResponse,
  AnnotationsResponse,
  AssetResponse,
} from "./types";

/** Resolve the engine API base. Default same-origin `"/api"`. For a SPLIT deploy ? SPA on
 *  Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel (HOSTING_PLAN.md §2) ?
 *  set `VITE_API_BASE` at build time to the engine's API base **including the `/api` path**
 *  (e.g. `https://api.khelsutra.guru/api`), and lock the engine's `RALLY_CORS_ALLOW_ORIGINS` to the
 *  Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed so
 *  `` `${base}/process` `` never double-slashes. */
export function apiBase(): string {
  return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
}

const BASE = apiBase();

export class ApiError extends Error {
  readonly cid?: string;
  readonly status?: number;
  readonly body?: string;
  constructor(message: string, opts: { cid?: string; status?: number; body?: string; cause?: unknown } = {}) {
    super(message);
    this.name = "ApiError";
    this.cid = opts.cid;
    this.status = opts.status;
    this.body = opts.body;
    if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
  }
```

```
}

interface RequestOpts {
  method?: string;
  body?: string;
  headers?: Record<string, string>;
  /** Correlation id to thread through this call; auto-generated if absent. */
  cid?: string;
}

async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
  const cid = opts.cid ?? newCorrelationId();
  const url = `${BASE}${path}`;
  const method = opts.method ?? "GET";
  log("info", "api.request", { cid, method, url });

  let res: Response;
  try {
    res = await fetch(url, {
      method,
      body: opts.body,
      headers: { "content-type": "application/json", "x-correlation-id": cid, ...(opts.headers ?? {}) },
    });
  } catch (err) {
    // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
    log("error", "api.network_error", { cid, method, url, error: String(err) });
    throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
  }

  if (!res.ok) {
    const body = await res.text().catch(() => "");
    log("error", "api.http_error", { cid, method, url, status: res.status, body: body.slice(0, 500) });
    throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status: res.status, body });
  }

  const data = (await res.json()) as T;
  log("info", "api.response", { cid, method, url, status: res.status });
  return data;
}

/** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host) for
 *  processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-fails 400
 *  (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as ApiError. */
export async function processVideo(
  videoPath: string,
  cid?: string,
  locale?: string,
  compute?: string,
): Promise<ProcessResponse> {
  // `locale` (the UI language) is recorded for PMF analytics (which markets use the tool). `compute`
  // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the server
  // default. Both optional + omitted when absent (JSON.stringify drops undefined).
  return request<ProcessResponse>("/process", {
    method: "POST",
    body: JSON.stringify({ video_path: videoPath, locale, compute }),
    cid,
  });
```

```
}

/** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
  return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid });
}

/** List indexed videos, newest first ? the "my videos" library. GET /api/videos [main.py:447].
 *  Reload-survival: a past match can be reopened without re-processing. Returns [] on an empty box. */
export async function listVideos(cid?: string): Promise<VideoSummary[]> {
  const data = await request<VideosResponse>("/videos", { method: "GET", cid });
  return data.videos ?? [];
}

/** The source-video stream URL for annotation playback. GET /api/source/{video_id} (P8). The engine
 *  serves a frame-identical proxy; labels stay keyed to the master's MD5 (STUDIO_MEDIA_LIFECYCLE D4). */
export function sourceUrl(videoId: string): string {
  return `${BASE}/source/${encodeURIComponent(videoId)}`;
}

/** Persist finished rally labels as the annotator CSV. POST /api/annotations {video_id, csv} (P8).
 *  Sent as the byte-compatible CSV (serializeRows) so the engine writes it verbatim into the corpus. */
export async function postAnnotations(videoId: string, csv: string, cid?: string): Promise<AnnotationsResponse> {
  return request<AnnotationsResponse>("/annotations", {
    method: "POST",
    body: JSON.stringify({ video_id: videoId, csv }),
    cid,
  });
}

/** Store a video into the user's chosen medium + register it as a library asset (P4, D14).
 *  POST /api/assets {uploaded_path? | source_uri?, medium_uri, sport?} ? 201 {video_id, stored_uri}.
 *  Exactly ONE source: an `uploadedPath` (a just-uploaded local file ? a durable MD5-keyed copy is
 *  put into the medium, the local upload kept as the working copy) OR a `sourceUri` (a video already
 *  at rest, registered as-is). Feed `stored_uri` into processVideo(). Fast-fails 400 (bad medium /
 *  both-or-neither source), 403 (hosted posture), 404 (missing file), 507 (no space), 502 (put) ?
 *  surfaced as ApiError. */
export async function createAsset(
  args: { uploadedPath?: string; sourceUri?: string; mediumUri: string; sport?: string },
  cid?: string,
): Promise<AssetResponse> {
  return request<AssetResponse>("/assets", {
    method: "POST",
    body: JSON.stringify({
      uploaded_path: args.uploadedPath,
      source_uri: args.sourceUri,
      medium_uri: args.mediumUri,
      sport: args.sport,
    }),
    cid,
```

```
  });
}

/** List a video's detected rallies. POST /api/query {video_id} ? {play_segments} [main.py:331]. */
export async function listRallies(videoId: string, cid?: string): Promise<RallySegment[]> {
  const data = await request<QueryResponse>("/query", {
    method: "POST",
    body: JSON.stringify({ video_id: videoId }),
    cid,
  });
  return data.play_segments ?? [];
}

// Compile-job poll cadence (mirrors useIngestJob): a long ffmpeg stitch must not block the request.
const COMPILE_POLL_MIN_MS = 1000;
const COMPILE_POLL_MAX_MS = 5000;
const COMPILE_POLL_BACKOFF = 1.5;
const COMPILE_POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' compile won't poll forever

/** Stitch selected rallies into one reel. ASYNC (#329): POST /api/compile returns 202 + a job_id;
 *  the ffmpeg stitch runs on the worker and we POLL GET /api/jobs/{id} for the reel ? a long stitch
 *  would otherwise exceed the ~100 s edge/proxy timeout (524) while the reel actually built. Resolves
 *  with the job's CompileResult on success; throws ApiError on a failed/timed-out compile.
 *  `locale` (the UI language) localizes the reel watermark engine-side (omitted when absent). */
export async function compileReel(
  videoId: string,
  segmentIds: number[],
  outputFilename: string,
  cid?: string,
  locale?: string,
): Promise<CompileResult> {
  const c = cid ?? newCorrelationId();
  const submitted = await request<ProcessResponse>("/compile", {
    method: "POST",
    body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename: outputFilename, locale }),
    cid: c,
  });
  return pollCompileJob(submitted.job_id, c);
}

/** Poll a submitted compile job to its terminal state. Returns the reel result on success; throws an
 *  ApiError for an engine job-failure, a success-shaped FAILURE (done but the stitch failed / no URL),
 *  or a poll timeout ? so the caller's existing classifyError/errorText mapping surfaces it like ingest. */
async function pollCompileJob(jobId: string, cid: string): Promise<CompileResult> {
  const startedAt = Date.now();
  let delay = COMPILE_POLL_MIN_MS;
  for (;;) {
    const job = await getJob(jobId, cid);
    if (job.status === "done") {
      const result = (job.result ?? {}) as CompileResult;
      if (result.status === "failed" || !result.download_url) {
        // A real stitch failure carries an engine message ? surface it as an `engine`
```

```
detail (status +
        // body set) so classifyError shows it verbatim, NOT the no-status "engine unreachable" fallback.
        const msg = result.message || "Highlight stitching failed.";
        throw new ApiError(msg, { cid, status: 502, body: msg });
      }
      return result;
    }
    if (job.status === "failed") {
      const msg = job.error || "Compile job failed.";
      throw new ApiError(msg, { cid, status: 502, body: msg });
    }
    if (Date.now() - startedAt > COMPILE_POLL_CEILING_MS) {
      throw new ApiError("Compile timed out.", { cid, status: 504, body: "Compile timed out." });
    }
    await new Promise((resolve) => setTimeout(resolve, delay));
    delay = Math.min(delay * COMPILE_POLL_BACKOFF, COMPILE_POLL_MAX_MS);
  }
}

/** The stream/download URL for a compiled reel. GET /api/highlights/{video_id}/{filename} [main.py:307]. */
export function highlightUrl(videoId: string, filename: string): string {
  return `${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
}

/** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
export async function getConfig(cid?: string): Promise<EngineConfig> {
  return request<EngineConfig>("/config", { method: "GET", cid });
}

/** What THIS box can do ? drives the first-run wizard. GET /api/capabilities (PACKAGING_PLAN P2c). */
export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
  return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
}

/** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide [main.py:362]. */
export function minShotCount(cfg: EngineConfig): number | undefined {
  return cfg.stitching?.min_shot_count;
}

// ?? chunked upload (backend/api/uploads.py) ??????????????????????????????????
// A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge proxies cap
// bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into /api/process.

/** Start a chunked upload. 400 = bad extension; 413 = over the size limit [uploads.py:64]. */
export async function uploadInit(filename: string, totalSizeBytes: number, cid?: string): Promise<UploadInitResponse> {
  return request<UploadInitResponse>("/upload/init", {
    method: "POST",
    body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
    cid,
  });
}

/** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,? [uploads.py:89]. */
export async function uploadPart(
```

```
  uploadId: string,
  index: number,
  chunk: Blob,
  cid?: string,
  signal?: AbortSignal,
): Promise<UploadPartResponse> {
  const c = cid ?? newCorrelationId();
  const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
  log("info", "api.request", { cid: c, method: "PUT", url });
  let res: Response;
  try {
    res = await fetch(url, {
      method: "PUT",
      body: chunk,
      headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
      signal,
    });
  } catch (err) {
    log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err) });
    throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
  }
  if (!res.ok) {
    const body = await res.text().catch(() => "");
    log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status, body: body.slice(0, 500) });
    throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status, body });
  }
  log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
  return (await res.json()) as UploadPartResponse;
}

/** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123]. */
export async function uploadComplete(uploadId: string, totalParts: number, cid?: string): Promise<UploadCompleteResponse> {
  return request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
    method: "POST",
    body: JSON.stringify({ total_parts: totalParts }),
    cid,
  });
}

/** Abort an in-flight upload (best-effort cleanup of the server-side partial) [uploads.py:147]. */
export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
  await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE", cid });
}

export interface UploadProgress {
  sentBytes: number;
  totalBytes: number;
}

/** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ? complete.
 *  On ANY failure or abort it best-effort DELETEs the server-side partial, then rethrows so the
 *  caller can classify it (the engine's 400/413 detail survives). Returns {path, video_id}.
 *  Note: a per-part 413 (over the per-part limit) aborts the upload server-side ? recovery is a
```

```
 *  fresh init (re-pick the file), which the UI's retry does. */
export async function uploadFile(
  file: File,
  opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void } = {},
): Promise<UploadCompleteResponse> {
  const cid = opts.cid ?? newCorrelationId();
  const init = await uploadInit(file.name, file.size, cid);
  // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math to NaN (which
  // would silently upload zero parts) ? fall back to the engine's 95 MB default.
  const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ? init.max_part_mb : 95;
  const partBytes = partMb * 1024 * 1024;
  const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
  log("info", "upload.start", { cid, filename: file.name, size: file.size, parts: totalParts, part_mb: init.max_part_mb });
  let sent = 0;
  try {
    for (let i = 0; i < totalParts; i++) {
      if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
      const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
      await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
      sent += chunk.size;
      opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
    }
    const done = await uploadComplete(init.upload_id, totalParts, cid);
    log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size: done.size_bytes });
    return done;
  } catch (err) {
    log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error: String(err) });
    await uploadAbort(init.upload_id, cid).catch(() => {});
    throw err;
  }
}
```

9. assistant (2026-07-02T02:47:48.044Z)

Now let me check the engine's API surface to compare against the SPA client.

10. user (2026-07-02T02:47:49.626Z)

```
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
backend:
__pycache__
annotations
api
assets
audio
billing.py
compute
config
detectors
eval
features
flywheel.py
infrastructure
main.py
pipeline
providers
```

```
reporting
results.py
sports
stitcher
storage
tools
ui
utils
worker

backend/api:
__init__.py
__pycache__
auth.py
job_worker.py
static
uploads.py
```

11. user (2026-07-02T02:47:50.929Z)

```
backend\api\uploads.py:70:@router.post("/api/upload/init")
backend\api\uploads.py:111:@router.put("/api/upload/{upload_id}/part/{index}")
backend\api\uploads.py:157:@router.post("/api/upload/{upload_id}/complete")
backend\api\uploads.py:188:@router.delete("/api/upload/{upload_id}")
backend\main.py:258:@app.get("/", response_class=HTMLResponse)
backend\main.py:379:@app.get("/api/config")
backend\main.py:391:@app.get("/api/health")
backend\main.py:419:@app.get("/api/version")
backend\main.py:429:@app.get("/api/capabilities")
backend\main.py:521:@app.get("/api/videos")
backend\main.py:537:@app.get("/api/reels")
backend\main.py:570:@app.get("/api/reels/{filename}")
backend\main.py:623:@app.get("/api/source/{video_id}")
backend\main.py:663:@app.post("/api/annotations")
backend\main.py:727:@app.post("/api/assets", status_code=201)
backend\main.py:1355:@app.post("/api/process", status_code=202)
backend\main.py:1431:@app.get("/api/jobs/{job_id}/cost")
backend\main.py:1446:@app.get("/api/videos/{video_id}/cost")
backend\main.py:1457:@app.get("/api/jobs/{job_id}")
backend\main.py:1492:@app.get("/api/highlights/{video_id}/{filename}")
backend\main.py:1523:@app.post("/api/query")
backend\main.py:1594:@app.post("/api/compile", status_code=202)
```

12. assistant (2026-07-02T02:48:02.661Z)

Endpoints all exist. Now check response-shape drift and the bundle-sync state.

13. user (2026-07-02T02:48:03.603Z)

```
// Types mirror the engine's verified contract (badminton-highlight-indexer).
// Anchors: backend/infrastructure/database.py:124-135 (play_segments),
// backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).

export interface RallyMetadata {
  /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some detectors. */
  shot_count?: number;
  /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
  shot_count_confidence?: string;
  detector?: string;
  [key: string]: unknown;
}

export interface RallySegment {
```

```
  /** The integer the UI selects and passes to /api/compile as segment_ids. */
  id: number;
  start_time: number;
  end_time: number;
  duration: number;
  /** motion-segmenter FABRICATION (random) ? DO NOT surface (see PER_RALLY_SELECTION.md). */
  server: string | null;
  /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
  receiver: string | null;
  metadata: RallyMetadata;
  /** motion-only fabricated sub-events ? DO NOT surface. */
  events?: unknown[];
}

export interface QueryResponse {
  play_segments: RallySegment[];
}

/** One row of the indexed-video library. GET /api/videos [backend/main.py:447] returns the most
 *  recent first so the console survives a reload. Every field but `video_id` is optional/defensive. */
export interface VideoSummary {
  /** The MD5 join key (the contract field). Optional defensively ? an older/regressed engine may
   *  send only the raw `id`; read it through `lib/video.ts::videoId`, never directly. */
  video_id?: string;
  /** Raw DB fields an engine may send instead of the enriched contract (resilience fallbacks). */
  id?: string;
  filepath?: string | null;
  match_name?: string | null;
  local_path?: string | null;
  fps?: number | null;
  resolution?: string | null;
  created_at?: string | null;
}

export interface VideosResponse {
  videos: VideoSummary[];
}

/** POST /api/annotations ? persist finished rally labels for a video as the annotator CSV (P8).
 *  The engine writes the CSV verbatim (byte-compatible with the pipeline), keyed by the file MD5. */
export interface AnnotationsResponse {
  status: string;
  num_rows?: number;
  csv_uri?: string;
  message?: string;
}

export interface CompileResponse {
  status: string;
  /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
  filepath: string;
}

/** The terminal RESULT of an async reel compile (#329): the job's `result` payload, surfaced by
 *  GET /api/jobs/{id} once the stitch finishes. `status` is the pipeline verdict ("success" |
```

```
 *  "failed"); a "failed" result carries a `message`. The SPA builds the download URL with
 *  highlightUrl() (split-origin-safe), but the engine also returns a same-origin `download_url`. */
export interface CompileResult {
  status: string;
  filepath?: string;
  download_url?: string;
  message?: string;
  video_id?: string;
}

export interface EngineConfig {
  stitching?: {
    /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it. */
    min_shot_count?: number;
    [key: string]: unknown;
  };
  [key: string]: unknown;
}

/** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do, rendered by
 *  the first-run wizard (PACKAGING_PLAN.md P2c). Presence-bools only (never the Gemini key value).
 *  Every field optional/defensive: a missing field degrades the wizard gracefully. */
export interface EngineCapabilities {
  api_version?: number;
  engine_version?: string;
  segmenters?: string[];
  default_segmenter?: string;
  ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?: string };
  gpu?: { nvidia_smi_present?: boolean };
  /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step. */
  gemini_key_present?: boolean;
  wasb_weights_present?: boolean;
  /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch (a Cloud Run
   *  GPU Job)? The ingest compute-picker shows the "in the cloud" option only when true. */
  deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
  /** Which storage media THIS box can store INTO (P3). The medium picker renders only `usable`
   *  backends; `free_gb` is `null` for a cloud bucket (no readable capacity). Absent on older engines
   *  ? the SPA degrades to local-only (no picker). */
  storage?: StorageCapabilities;
  [key: string]: unknown;
}

/** One storable medium reported by GET /api/capabilities (P3). Presence/usable bools only ? no
 *  secrets. `id`/`kind` are the backend name (local/gcs/s3/gdrive). */
export interface StorageBackendInfo {
  id: string;
  kind: string;
  label: string;
  usable: boolean;
  /** Free space in whole GB, or `null` when unknowable (a cloud bucket). */
  free_gb?: number | null;
}

```

```
export interface StorageCapabilities {
  default_medium?: string;
  backends?: StorageBackendInfo[];
}

/** POST /api/assets ? 201 (P4). The video was stored into the chosen medium and registered as a
 *  library asset keyed by the whole-file MD5 (`video_id`). `stored_uri` is the durable location to
 *  feed into POST /api/process (a bucket/Drive URI, or a local path for "this computer"). */
export interface AssetResponse {
  video_id: string;
  /** The medium backend the asset lives in (local | s3 | gcs | gdrive). */
  medium: string;
  /** The durable, MD5-keyed location ? pass as video_path to processVideo(). */
  stored_uri: string;
  /** True when a durable copy was made (uploaded ? remote medium); false when registered in place. */
  copied: boolean;
  sport?: string | null;
  status?: string;
}

/** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET /api/jobs/{job_id}. */
export interface ProcessResponse {
  job_id: string;
  /** Always "queued" at submit. */
  status: string;
  /** Relative poll path, e.g. "/api/jobs/<id>". */
  poll: string;
}

/** Chunked upload (backend/api/uploads.py). The browser uploads a local file in sequential parts
 *  ? max_part_mb, then `complete` returns a server-host `path` to feed straight into POST /api/process. */
export interface UploadInitResponse {
  upload_id: string;
  /** Per-part ceiling in MB ? slice the file into parts this big. */
  max_part_mb: number;
  next_part: string;
}
export interface UploadPartResponse {
  upload_id: string;
  received_parts: number;
  received_bytes: number;
}
export interface UploadCompleteResponse {
  /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
  path: string;
  video_id: string;
  size_bytes: number;
}

/** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
 *  `status` = the EXECUTION state; `result.status` = the pipeline VERDICT (success|failed).
 *  A `done` job whose `result.status === "failed"` (or with no video_id) is a success-shaped
 *  FAILURE ? the UI must not treat it as a load. */
export interface JobStatus {
  job_id: string;
  status: "queued" | "running" | "done" | "failed";
```

```
    /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a percentage. */
    progress?: string | null;
    /** The canonical video_id, surfaced top-level on success (also in result.video_id). */
    video_id?: string | null;
    /** Execution error (worker crash / unexpected) when status === "failed". */
    error?: string | null;
    result?: { status?: string; video_id?: string; message?: string } | null;
  }

```

14. user (2026-07-02T02:48:05.474Z)

```
/c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/static/:
total 52
drwxr-xr-x 1 avidu 197609     0 Jun 25 16:24 .
drwxr-xr-x 1 avidu 197609     0 Jul  2 08:13 ..
-rw-r--r-- 1 avidu 197609 17913 Jun 25 16:24 app.js
-rw-r--r-- 1 avidu 197609  8743 Jun  7 14:14 index.html
-rw-r--r-- 1 avidu 197609 13418 Jun  7 14:14 style.css

/c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/ui/:
total 18
drwxr-xr-x 1 avidu 197609   0 Jun 25 16:24 .
drwxr-xr-x 1 avidu 197609   0 Jul  2 08:13 ..
-rw-r--r-- 1 avidu 197609 683 Jun 25 16:24 THIRD_PARTY_NOTICES.md
drwxr-xr-x 1 avidu 197609   0 Jun 25 16:24 assets
-rw-r--r-- 1 avidu 197609 401 Jun 25 16:24 index.html
-rw-r--r-- 1 avidu 197609 273 Jun 25 16:24 ui_version.json
index-Jp9DK5o0.css
index-ukuS9wxa.js
noto-sans-devanagari-devanagari-wght-normal-DWI_L5Cu.woff2
noto-sans-devanagari-latin-ext-wght-normal-CLkupaxV.woff2
noto-sans-devanagari-latin-wght-normal-Bxfr4UJH.woff2
noto-sans-kannada-kannada-wght-normal-DuwB-P8d.woff2
noto-sans-kannada-latin-ext-wght-normal-MdBBF41x.woff2
noto-sans-kannada-latin-wght-normal-DeLZCgFT.woff2
```

15. user (2026-07-02T02:48:14.699Z)


```
{
  "spa_repo": "https://github.com/avidullu/khelsutra.git",
  "spa_commit": "ed33a515a4c68cb42134e8c7d4514df2addb9f54",
  "engine_api_version": 1,
  "content_hash": "b0801e7a231d4849cb33d374840f92d7ab2aca3f6ba9088124bbdd674ff15851",
  "built_at": "2026-06-22T17:59:59Z"
}

```


16. user (2026-07-02T02:48:16.379Z)

```
f9eea14 2026-07-01 web: resilient video-summary readers + show fps in the library (#140)
fb2f561 2026-07-01 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
8667dbf 2026-07-01 P5: SPA storage-medium picker (store a video into your medium) (#137)
cc06c32 2026-07-01 feat(web): P8 annotate-in-UI ? AnnotatePanel over the vendored core (flagged) (#128)
9e9df72 2026-07-01 feat(web): keep the video name visible after open (persistent current-video label) (#127)
926b53a 2026-07-01 feat(web): vendor the rally-annotator annotation core (P8 groundwork) (#126)
39c1349 2026-07-01 feat(web): P6 ? "Your videos" library (reopen a stored video) (#124)
d0285cd 2026-07-01 test(e2e): single-origin mock engine + Playwright screencast harness (#122)
ed33a51 2026-06-22 feat(build): poll the async /api/compile job for the reel (#329) (#108)
f67dfe7 2026-06-22 feat(ingest): compute-picker ? choose local vs cloud GPU per run (item 3b)
```

(#104)
e2570e4 2026-06-22 docs: archive the Friendly & Multilingual tracker (engineering complete) (#93)
7ae9cee 2026-06-22 feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages) (#88)
---
f9eea14 2026-07-01 22:08:00 +0530

17. user (2026-07-02T02:48:31.279Z)

```
# PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from an
# arbitrary directory finds the shipped assets instead of creating a stray ./backend/api/static.
# (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
_STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
try:
    os.makedirs(_STATIC_DIR, exist_ok=True)
except OSError:
    # The dir ships with the package (a no-op create); guard against a read-only install
    # tree (system-installed Python) since exist_ok suppresses FileExistsError, not PermissionError.
    pass
app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")

# P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed snapshot
# (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only if it's
# absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so no SPA change
# is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets is enough
# (no client-side path routing ? no history-fallback catch-all, which would also be a route-order hazard).
_BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
_UI_DIR = (
    _BUNDLED_UI_DIR
    if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
    else _STATIC_DIR
)
_UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
if os.path.isdir(_UI_ASSETS_DIR):
    app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")


@app.get("/", response_class=HTMLResponse)
def serve_index():
    index_path = os.path.join(_UI_DIR, "index.html")
    if os.path.exists(index_path):
        with open(index_path, "r", encoding="utf-8") as f:
            return f.read()
    return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"


def _bundled_ui_version() -> Optional[Dict[str, Any]]:
    """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is bundled.
    Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew guard)."""
    path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
    try:
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    except (OSError, ValueError):
```

```
        return None


# Global variables initialized at startup
db_instance: Optional[Database] = None
global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
# P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if it were
# GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI init).
_instance_lock: Optional[Any] = None

# Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md P3)
# now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
# `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams THROUGH
# this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
```

18. user (2026-07-02T02:48:32.138Z)

```
            resolution = f"{w}x{h}" if w and h else None
            return (fps if isinstance(fps, (int, float)) else None), resolution
    return None, None


def _video_summary(row: Dict[str, Any]) -> Dict[str, Any]:
    """Map a DB video row to the API/SPA contract. ADDITIVE ? keeps the raw ``id``/``filepath`` (an
    existing test + any old client rely on them) and adds the fields the SPA library + the persistent
    "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a friendly
    ``match_name`` (so an uploaded video shows its filename, not a generic fallback), ``local_path``,
    ``created_at`` (from ``processed_at``), and ``fps``/``resolution`` (from the stored ffprobe
    validation details) so the library cards can show them. Mirrors the mock engine's shape so mock and
    real agree."""
    fp = row.get("filepath") or ""
    fps, resolution = _video_dims(row.get("validation_results"))
    return {
        **row,
        "video_id": row.get("id"),
        "match_name": _friendly_video_name(fp),
        "local_path": fp,
        "created_at": row.get("processed_at") or row.get("created_at"),
        "fps": fps,
        "resolution": resolution,
    }


@app.get("/api/videos")
def list_videos(request: Request, limit: Optional[int] = None):
    """List indexed videos (most recent first) so the console survives a reload ? previously only a
    single-row get existed (PACKAGING_PLAN.md P1). Rows are enriched to the SPA contract
    (``video_id`` + a friendly ``match_name`` derived from the filename) so the library shows real
    names and the persistent "current video" label survives an upload ? see ``_video_summary``."""
    return {"videos": [_video_summary(v) for v in request.app.state.db.list_videos(limit=limit)]}
```

```


def _reels_dir(cfg) -> str:
    return getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"


_REEL_EXTS = (".mp4", ".mov")


@app.get("/api/reels")
def list_reels(request: Request):
    """List compiled highlight reels present in the output dir, newest first, with a
download URL.
```

19. user (2026-07-02T02:48:43.952Z)

```
README.md
__pycache__
bundle_ui.sh
doctor.py
install.py
nightly_email.py
nightly_reelcount.py
nightly_regression.ps1
nightly_regression.py
register_nightly_task.ps1
run_phase3_eval.py
run_process_and_stitch.py
uninstall.py
verify_compute_path.py
---
test.yml
---
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/deploy:
DEPLOY_ALPHA.md
GPU_VM_SERVING.md
cloudflared.config.example.yml
droplet
stage_from_bucket.sh

/c/Users/avidu/Projects/khelsutra-guru/khelsutra/docs:
ANNOTATE_AND_TRAIN.md
DRY_RUN_CUJ.md
LOCAL_APPLIANCE_ARCHITECTURE.md
MULTILINGUAL_EXPANSION.md
PER_RALLY_SELECTION.md
PORTABILITY.md
README.md
REQUEST_ACCESS_FORM.md
STUDIO_MEDIA_LIFECYCLE.md
archives
```

20. user (2026-07-02T02:48:44.685Z)

```
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

// The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
// proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
// (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local engine
// (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if needed.
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
```

```
    proxy: {
      "/api": { target: "http://localhost:8000", changeOrigin: true },
    },
  },
  test: {
    // Default env is node (the pure lib/parser tests); the i18n render test opts into jsdom via
a
    // `// @vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
    environment: "node",
    include: ["src/**/*.test.{ts,tsx}"],
    // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files no-op.
    setupFiles: ["src/test/setup.ts"],
    // Coverage floor (CI runs `npm run test:coverage`): a merge can't DROP coverage below these
    // thresholds, so untested new code is caught. `all` counts every source file (not only the
ones a
    // test imported), so deleting a test surfaces here too. Excludes are non-logic files: the
app
    // entry/bootstrap, type-only declarations, and the test harness itself.
    coverage: {
      provider: "v8",
      reporter: ["text", "text-summary"],
      all: true,
      include: ["src/**/*.{ts,tsx}"],
      exclude: [
        "src/**/*.test.{ts,tsx}", // the tests
        "src/test/**", // jsdom localStorage shim (test harness, not shipped logic)
        "src/main.tsx", // app entry: mounts <App/> + bundles fonts ? nothing to unit-test
        "src/api/types.ts", // type-only declarations (no runtime code)
        "src/annotator/**", // VENDORED from rally-annotator (MIT) ? fully tested upstream; a
smoke test guards the copy
        "**/*.d.ts",
      ],
      // Floor set just below current (stmts/lines 93.3, branches 88.1, funcs 85.9) ? a real
ratchet
      // that fails CI on a meaningful drop, with a little slack so it doesn't flap. Ratchet
upward as
      // coverage grows. (functions stays a touch looser: App.tsx is the engine-orchestrating
shell,
      // costly to exercise fully in unit tests.)
      thresholds: {
        statements: 92,
        branches: 87,
        functions: 84,
        lines: 92,
      },
    },
  },
});
```

21. user (2026-07-02T02:48:53.430Z)

```
name: test
on:
  push:
    branches: [master]
  pull_request:

jobs:
  site:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: site
    steps:
```

```
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v6
        with:
          node-version: "24" # node:sqlite is unflagged; server.test.js boots the VM host
          cache: "npm"
          cache-dependency-path: site/package-lock.json
      # jsdom is a TEST-ONLY devDep (the behavioral landing-page UX tests run the real inline
      # script in a DOM); the Cloudflare Worker + VM-host RUNTIMES stay dependency-free. `npm test`
      # runs ALL paths: Worker+D1 logic (index.test.js), the booted VM host (server.test.js), the
      # static content contract (assets.test.js), and the UX behaviour (ux.dom.test.js).
      - run: npm ci
      - run: npm test
      # Syntax-check the ops shell scripts (provision/teardown/smoke/parity) so a broken
      # script can't ship to the runbook. `bash -n` parses without executing.
      - name: shell scripts parse-check
        run: for f in scripts/*.sh; do echo "checking $f"; bash -n "$f"; done

  web:
    # Operator-console SPA (web/): typecheck + unit tests (with a coverage floor) + production build
    # gate the merge. `test:coverage` fails if coverage drops below the thresholds in vite.config.ts,
    # so a merge can't silently regress test coverage.
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: web
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v6
        with:
          node-version: "24"
          cache: "npm"
          cache-dependency-path: web/package-lock.json
      - run: npm ci
      - run: npm run typecheck
      - run: npm run test:coverage
      - run: npm run build

  layout:
    # Real-browser layout-regression guard. The other suites run in jsdom, which has NO layout engine
    # (getBoundingClientRect() is all zeros) and therefore cannot see header overflow / wrapping /
    # clipping ? the responsive-nav bug that recurred. This drives a headless Chromium over the real
    # pages at a spread of viewport widths x all 6 locales and asserts the header never distorts.
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: site
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v6
        with:
          node-version: "24"
          cache: "npm"
          cache-dependency-path: site/package-lock.json
      - run: npm ci
      - run: npx playwright install --with-deps chromium
```

```
            - run: npm run test:layout
            # Cross-locale render guard: drives the real pages in every locale and asserts the live DOM
            # matches the catalog (no raw keys / no shim regression / right lang / no console errors).
            - run: npm run test:render
            # Accessibility guard: axe-core WCAG 2.1 A/AA over every page × representative locales; fails on
            # any serious/critical violation (jsdom has no contrast/layout engine to catch these).
            - run: npm run test:a11y

```

22. user (2026-07-02T02:48:54.554Z)

```
deploy/droplet:
README.md
config.json.example
engine.env.example
khelsutra-engine.service

locales:
da
en
es
hi
id
kn

site/public:
capture.html
flyer-da.html
flyer-es.html
flyer-hi.html
flyer-id.html
flyer-kn.html
flyer.html
i18n.js
img
index.html
locales
og.png
own-model.html
privacy.html
robots.txt
video

site/scripts:
copy-locales.mjs
droplet-smoke.sh
provision-vm.sh
teardown-vm.sh

site/src:
a11y.browser.mjs
assets.test.js
deploy-config.test.js
i18n.shim.test.js
index.js
index.test.js
layout.browser.mjs
locales-drift.test.js
notify.js
render.browser.mjs
server.test.js
```

store.js
ux.dom.test.js

23. user (2026-07-02T02:49:03.300Z)

1 # Deploy KhelSutra Studio for alpha ? a shareable page, no SSH
2
3 > **Status:** `LIVE` (deploy live-verified on the droplet 2026-06-22 ? `deploy/droplet/README.md`; only a rotated Gemini key + a browser OTP login remain to exercise the upload?reel CUJ) · **Topology:** split ? SPA on Cloudflare Pages + engine on an `api.*` cloudflared tunnel, Cloudflare Access over both (the engine's `HOSTING_PLAN.md` §2 alpha plan). · **Cost:** ~$0/mo standing (Pages + Tunnel + Access are free ?50 users) + Gemini per-run (~$0.05/match).
4
5 This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a **page you can share with alpha testers** ? they open `app.khelsutra.guru`, sign in (email allowlist), point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.
6
7 > **Outgrown the droplet?** To move the live engine onto a **GCP L4 GPU VM** (more CPU/RAM for ffmpeg
8 > + optional WASB GPU detection) and cut the tunnel over to it, see **[`GPU_VM_SERVING.md`](GPU_VM_SERVING.md)**
9 > ? same tunnel + Access, no DNS change, droplet kept as fallback (~$500?600/mo while the VM runs).
10
11 ## 0. The shape
12
13 ```
14 tester browser
15   ? https
16   ?
17 Cloudflare edge (free): DNS · TLS · Access (email-OTP allowlist) in front of BOTH hostnames
18   ??? app.khelsutra.guru ? Cloudflare Pages   (the web/ SPA; built with VITE_API_BASE)
19   ??? api.khelsutra.guru ? cloudflared Tunnel (outbound-only) ?? 127.0.0.1:8000 on YOUR box
20   ? FastAPI engine
21   ? (Gemini segmenter; GPU optional)
22   ? async jobs, SQLite, output/ on disk
23 ```
24
25 - **No inbound ports.** `cloudflared` dials out; the engine stays bound to `127.0.0.1:8000`. The box's home/public IP is never exposed.
26 - **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also* verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a direct hit to the tunnel can't spoof identity.
27 - **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ? the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's already env-driven).
28
29 > **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA *and* the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see `LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose (stable Pages-hosted page + a thin API tunnel).
30
31 ---
32
33 ## 1. Prerequisites
34
35 - A **box** to run the engine: the CPU droplet `168.144.126.176` (Gemini-only ? no GPU; fine for alpha) **or** your Windows+WSL GPU box (real WASB). `ffmpeg` + Python 3.8+ installed;

the `badminton-highlight-indexer` engine checked out.
- A **Cloudflare account** with the `khelsutra.guru` zone (you already run the marketing site there).
- `cloudflared` installed on the box (`https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/downloads/`).
- A **fresh** `GEMINI_API_KEY` (rotate the chat-shared one ? it uploads downsized chunks to Google; this is a paid + cloud action).

---

2. Engine (the `api.` side)

2.1 Install (with the auth extra)
```bash
cd badminton-highlight-indexer
pip install -e .[auth]          # PyJWT[crypto] ? REQUIRED when edge auth is on, else every authenticated request is rejected 401
```

2.2 The safe-exposed config
Create / edit `config.json` on the box. **Where it must live:** the engine reads `$RALLY_APP_HOME/config.json` if `RALLY_APP_HOME` is set, else `./config.json` relative to *the directory you launch the server from* ? **not** necessarily the repo root. If the engine reads a different (or absent) file it silently falls back to all-defaults, which means **edge auth is OFF**. To make writer and reader agree by construction, write it with `python -m backend.main --setup` (it targets the exact path the server reads) and edit *that* file:
```jsonc
{
  "security": {
    "edge_auth_enabled": true,                       // turn the CF-Access JWT verify ON
    "allowed_video_root": "/srv/khelsutra/incoming",  // the path-jail ? REQUIRED when exposed
    "upload_dir": "/srv/khelsutra/incoming/uploads",  // MUST sit UNDER allowed_video_root (see §7)
    "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
    "cf_access_aud": "<the CF Access application AUD>"  // filled in §4
  },
  "indexing": {
    "default_segmenter": "gemini"  // ? CPU box MUST set this ? the shipped default `yolo_hybrid` needs torch/GPU, and the SPA sends no segmenter, so every job fails "No module named 'torch'" without it
  }
}
```
And export the env (a `.env` or your shell):
```bash
export GEMINI_API_KEY="<your-rotated-key>"
export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru"   # lock CORS to the SPA origin
export RALLY_ALLOWED_HOSTS="api.khelsutra.guru,localhost,127.0.0.1"  # REQUIRED ? see ? below; without it EVERY tunneled request 400s (loopback added so on-box /api/health checks still pass ? setting this var REPLACES the loopback-only default)
export RALLY_BIND_HOST="127.0.0.1"                             # keep the loopback bind (cloudflared dials 127.0.0.1:8000)
```

> **? Two env vars you MUST set, or the tunnel is dead-on-arrival.** With `edge_auth_enabled: true`, the engine (a) flips its bind to `0.0.0.0` and (b) keeps the DNS-rebinding **Host-header guard** at its loopback-only default ? so it boots "healthy" but then **`400`s every request `cloudflared` forwards with `Host: api.khelsutra.guru`** (including the §6 `/api/health` check). The boot only *warns* about this (`EXPOSED_HOST_ALLOWLIST_LOOPBACK`), it does not stop. Set **`RALLY_ALLOWED_HOSTS="api.khelsutra.guru,localhost,127.0.0.1"`** (the load-bearing fix ? the Host allowlist is independent of the bind; the loopback entries keep the on-box `/api/health` check passing, since setting this var REPLACES the loopback-only default) and **`RALLY_BIND_HOST="127.0.0.1"`** (so the bind stays loopback as §0/§6 claim; `cloudflared`

dials loopback anyway). *Verified against engine `origin/master`: with the allowlist unset, `GET /api/health` with `Host: api.khelsutra.guru` ? `400`; with it set ? `200`.*

> **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails fast** if `edge_auth_enabled` is on but the `allowed_video_root` jail or `cf_team_domain`/`cf_access_aud` is missing, and **warns** if `PyJWT` isn't installed. So a half-configured exposure can't silently come up ? fix what it prints and restart.

2.3 Run
```bash
python -m backend.main --server     # binds 127.0.0.1:8000 ? kept loopback by RALLY_BIND_HOST (§2.2); with edge_auth ON and that var unset it would bind 0.0.0.0 instead
```
Keep it alive with a process manager (`systemd` on the droplet, mirroring `site/scripts/provision-vm.sh`'s unit; or `pm2`/`nssm` on Windows).

---

3. The SPA on Cloudflare Pages (the `app.` side)

The SPA ships as a **static Pages site**, git-connected like the marketing site (auto-deploy on merge to `master`).

1. **Cloudflare dashboard ? Workers & Pages ? Create ? Pages ? Connect to Git** ? pick the `khelsutra` repo.
2. **Build settings:**
   - **Root directory:** `web`
   - **Build command:** `npm run build`
   - **Build output directory:** `web/dist`
   - **Environment variable:** `VITE_API_BASE = https://api.khelsutra.guru/api` ? this is what points the SPA at the engine tunnel (PR #64).
3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.

> Every push to `master` now rebuilds + redeploys the SPA. To preview a change first, Pages builds a per-PR preview URL.

---

4. Cloudflare Access (gate BOTH hostnames)

1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
2. **Application domain:** add **both** `app.khelsutra.guru` **and** `api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester signs in once and both the SPA and its API calls are authorized).
3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50 users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
4. **Copy the application `AUD` tag** ? paste into the engine's `config.json` `security.cf_access_aud` (§2.2), and set `cf_team_domain` to `https://<your-team>.cloudflareaccess.com`. Restart the engine.
5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to `api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass options requests to CORS preflight?** (or add an explicit OPTIONS bypass), else the preflight is redirected to the login page and every API call fails. The engine's CORS middleware (`allow_credentials=false`, no cookies) then answers the preflight.

---

5. The tunnel (engine ? Cloudflare)

```bash
cloudflared tunnel login                                 # browser auth to your zone
cloudflared tunnel create khelsutra-studio               # note the <TUNNEL_UUID>
# copy deploy/cloudflared.config.example.yml ? /etc/cloudflared/config.yml, fill <TUNNEL_UUID>
cloudflared tunnel route dns khelsutra-studio api.khelsutra.guru
```

```
cloudflared tunnel run khelsutra-studio                   # or install as a service: `cloudflared service install`
```
See `deploy/cloudflared.config.example.yml` for the ingress (only `api.khelsutra.guru ? http://127.0.0.1:8000`).

---

6. Verify (the posture test + the CUJ)

Exposure posture (must hold ? this is what makes it safe to share):
- From OUTSIDE the box: `curl --max-time 5 http://<box-public-ip>:8000/api/health` ? **refused / timeout** (the engine is `127.0.0.1`-bound; no inbound port). ? The box is not directly reachable.
- `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to the Access login** (not a 200). ? The gate is in front.
- Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the SPA loads and `/api/health` returns 200 (same session cookie). ?

The shareable CUJ (what a tester does): open `app.khelsutra.guru` ? sign in ? **upload a video from their device** (the SPA chunked-upload lands it in `upload_dir`, inside the jail) ? **Process** ? watch progress ? pick rallies ? **Build** ? **download the reel**. All from the page. *(Pasting a `gs://?` URI does NOT work on this box ? see §7: the `allowed_video_root` jail rejects every non-local URI. Browser upload is the alpha path.)*

---

7. Honest caveats / deferred

- **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole). Keep `upload_dir` inside the jail (§2.2).
- **`gs://` / `s3://` / `gdrive://` URIs are REJECTED on the exposed box.** Setting `allowed_video_root` (required to expose, §2.2) path-jails `/api/process` to *local* paths under that root; the engine resolves a `gs://?` URI to a local path that escapes the jail ? `400` (pinned by engine `tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri`). Cloud-URI ingest works **only** when `allowed_video_root` is unset (the local single-user posture ? the opposite of this runbook). For alpha, the ingest path is **browser upload** (it lands inside the jail). *(Scheme-exempting recognized buckets from the jail would be an engine change, not a config flip.)*
- **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app JWT verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
- **Cross-origin reel download:** the reel streams from `api.` with `Content-Disposition: attachment`, so it downloads even cross-origin ? but verify the filename in your browser during the CUJ (cross-origin `<a download>` filename hints are ignored; the header is the authority).
- **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large matches upload as ~95 MB chunks (the SPA chunked-upload path honours the engine's `max_part_mb`, default **95**; lower `security.max_part_mb` to shrink them). `gs://` bucket pre-staging is **not** an option on this box (see the jail caveat above).
- **Box offline = engine offline.** The Pages SPA stays up and shows a friendly "engine offline" banner (`/api/health` ping); processing resumes when the box is back. Beta removes this with a queue (`HOSTING_PLAN.md` §3).
- **Gemini = paid + uploads footage to Google.** Keep the key rotated; use it only for clips you're OK sending.

---

8. Teardown
`cloudflared tunnel cleanup khelsutra-studio` + delete the tunnel; remove the Pages custom domain; disable the Access application; stop the engine service. The box's `output/` + `uploads/` retain originals ? sweep them per your retention policy before any wider exposure.

Changelog

153   - 2026-06-22 ? **correctness pass** (adversarial verify against engine `origin/master` + a hermetic repro). Fixed two would-break-the-deploy gaps: (1) **`RALLY_ALLOWED_HOSTS` was missing** ? with `edge_auth` on, the Host-header guard 400s every tunneled request until it's set (added it + `RALLY_BIND_HOST` to §2.2; repro confirmed 400?200); (2) **`gs://` ingest is rejected** by the `allowed_video_root` jail on the exposed box (dropped the `gs://` promise from §6/§7; alpha path = browser upload). Minor: 401-not-500 wording (§2.1), `config.json` location/`--setup` note (§2.2), bind comment (§2.3), ~95 MB chunk size (§7).
154   - 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared + Pages + CF Access) consuming the engine exposure-safety boot posture (engine #294) + the SPA `VITE_API_BASE` (#64). Pairs with `LOCAL_APPLIANCE_ARCHITECTURE.md` (single-origin alternative) + the engine `HOSTING_PLAN.md` §2.
155

24. user (2026-07-02T02:49:05.008Z)

Move KhelSutra Studio onto an L4 GPU VM (off the droplet)

Self-serve runbook to run the live engine (behind `api./app.khelsutra.guru`) on a **GCP L4 GPU VM**
instead of the CPU droplet, and **cut the Cloudflare tunnel over** so traffic lands on the VM. Lets
you switch detection between **Gemini** (cloud API, proven) and **WASB native** (uses the L4 GPU).

This extends [`DEPLOY_ALPHA.md`](DEPLOY_ALPHA.md) (which deployed the same engine on the droplet) ? it
reuses the droplet config templates in [`deploy/droplet/`](droplet/) and the GPU-VM scripts in the
engine repo (`badminton-highlight-indexer`: `deploy/gcp/`).

> **These are manual steps today.** Automating them with a one-shot **startup-script** you feed into
> the GCP console (`create_gpu_vm.sh` already injects a metadata startup-script ? today it only installs
> the Ops Agent) is backlogged: engine repo `docs/BACKLOG.md` ? *Batch / declarative orchestration*. The
> design hinge there is keeping the secrets (Gemini key / Access AUD / tunnel token) out of plaintext
> metadata.

> **Cost ??** A `g2-standard-4` (1×L4) bills **~$0.70?0.85/hr ? ~$500?600/month** while it runs. You
> keep it up, so this is an ongoing cost. **`bash deploy/gcp/teardown.sh delete` is the only way to
> stop billing** (a *stopped* VM still bills its disk). For bursty/intermittent load, a scale-to-zero
> **Cloud Run GPU *service*** would be far cheaper (~$0 idle) ? that's the backlogged "M7" and a bigger
> build; this VM is the fast path that works today.
>
> **Cutover is safe + reversible.** Bring the VM up and **verify it serving** before touching the
> tunnel; the droplet stays as a hot fallback. The tunnel is **token-managed** (ingress lives in the
> Cloudflare dashboard, not on the box), so moving it is just "run the same connector on the VM, then
> stop it on the droplet" ? **no DNS change, no new Cloudflare token, no CF-Access change.**

---

0. Prerequisites (gather these once)

- **gcloud** authenticated as the project owner (`gcloud auth login`) on the machine you run this from.

- **GPU quota** `GPUS_ALL_REGIONS ? 1` and the SA Ops-Agent roles ? see the engine repo
  `deploy/gcp/README.md` §2. (You already have L4 quota=1 from the nightly runs.)
- **The two secrets**, which you already have from the droplet deploy:
  - the **rotated Gemini API key** (Drive: `gemini.api_key.txt`) ? only needed for the Gemini path;
  - the **CF-Access AUD** + team domain ? already in the droplet's `/srv/khelsutra/config.json`
    (`security.cf_access_aud` / `cf_team_domain`); reuse the same values.
  - the **tunnel token** ? in the droplet's cloudflared unit (`systemctl cat cloudflared` ? the
    `--token ?` value). You'll copy it to the VM in §6.

====DROPLET-README====

Droplet quickstart ? KhelSutra alpha (the `api.` engine side)

The **filled-in, command-dense** version of `../DEPLOY_ALPHA.md` for the Linux CPU droplet (`168.144.126.176`, Gemini, **no GPU**). Values are pre-baked for **`app.khelsutra.guru` / `api.khelsutra.guru`**, team domain **`https://khelsutra.cloudflareaccess.com`**, allowlist **`avi.dullu@gmail.com`**, jail **`/srv/khelsutra/incoming`**. Read `../DEPLOY_ALPHA.md` for the *why*;
this is the *what to type*. Files here:

File	Goes to	Purpose
`config.json.example`	`/srv/khelsutra/config.json`	the `security` block (edge-auth + jail + CF team/AUD)
`engine.env.example`	`/etc/khelsutra/engine.env`	secrets + the **two `RALLY_*` exposure vars** + app-home
`khelsutra-engine.service`	`/etc/systemd/system/`	keeps the engine up (binds `127.0.0.1:8000`)
`../cloudflared.config.example.yml`	`/etc/cloudflared/config.yml`	the tunnel ingress (`api.` ? `127.0.0.1:8000`)

> **Cost reality:** Cloudflare Pages + Tunnel + Access are **free** (Access ? 50 users). **No $25/Pro
> plan.** When you enable Zero Trust you may be asked to add a card and "subscribe" ? pick the **Free**
> plan; you won't be charged under 50 users. Standing cost ? $0 + Gemini per run.

A. Engine on the box (one-time)

```bash

# 1. user + jail dirs (upload_dir MUST sit under allowed_video_root)

sudo useradd --system --create-home --home-dir /opt/khelsutra khelsutra || true
sudo mkdir -p /srv/khelsutra/incoming/uploads
sudo chown -R khelsutra:khelsutra /srv/khelsutra /opt/khelsutra

# 2. engine code + venv + the AUTH extra (REQUIRED with edge auth on, else every aut

#### 25. user (2026-07-02T02:49:17.418Z)

No matches found

#### 26. user (2026-07-02T02:49:17.886Z)

Found 11 files
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.test.ts
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\docs\DRY_RUN_CUJ.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\NEXT_STEPS.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\GPU_VM_SERVING.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\DEPLOY_ALPHA.md
```

```
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\droplet\engine.env.example
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\droplet\README.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\vite-env.d.ts
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\cloudflared.config.example.yml
```

27. user (2026-07-02T02:49:24.031Z)

```
# Next steps (live) ? KhelSutra brand / site

Prioritized "what to do next" for the **`khelsutra` product repo**. Refresh at each checkpoint.

**Where we are (updated 2026-07-01):** `khelsutra.guru` is **LIVE and fully shipped** ? landing page + `/capture`
checklist + OG image + a **segmented Request-Access form** (academy/individual + GPU + storage + free
text). On **Cloudflare** (Worker `hidden-king-khelsutra` + D1 `khelsutra-access`): the form persists to
D1 **and emails a summary to the owner on every registration** (Email Routing send binding ? verified
live). **Git auto-deploy is connected** (Workers Builds, root `site/`): merge to `master` ? deploys;
**CI gates every PR** ? three jobs: the full site `npm test` (Worker + VM host + assets + UX), the
`web/` SPA build + coverage floor, and a real-browser layout check. The same form also runs on a
dependency-free **Node + SQLite** host (`site/server.js`), **live + browsable on the DO droplet ?
http://168.144.126.176:8787** (systemd `khelsutra-site`). Shipped across ~90 merged PRs (the form,
6-language localization, the per-rally MVP, droplet portability). Docs:
`docs/REQUEST_ACCESS_FORM.md`, `docs/PORTABILITY.md`.

## Next product threads (owner ideas, 2026-06-20)
- **? DONE ? Friendly & Multilingual (ARCHIVED 2026-06-22; human gates cleared):** the product is
  non-tech-friendly and **fully localized to all 6 languages** (en/kn/hi/es/da/id) across both surfaces
  (`web/` SPA + `site/` marketing) ? including the home page (hero, all sections, the `#join` access form,
  footer), the **Privacy Policy** page, **per-page `<title>`s**, and **es/da/id academy flyers** (with
  localized SPA screenshots). The responsive header is fixed and a **headless-Chromium CI suite** guards
  layout, cross-locale render (live DOM == catalog), and **accessibility (axe-core WCAG 2.1 AA)**. Full
  record: **`docs/archives/FRIENDLY_AND_MULTILINGUAL.md`**. Post-archive PRs: #86 (own-model) · #87 (es/da/id
  site) · #105 (form+footer + render guard) · #106 (Privacy page) · #107 (a11y fixes + axe guard + page
  titles) · #109 (flyers). **Owner gates CLEARED:** native-speaker review of the AI-drafted catalogs ? +
  real-device font-glyph eyeball ?. **Remaining (small/owner):**
  - **Delta native-review** of the strings ADDED after the owner's sign-off (#105/#106/#107/#109 ? the
    form/footer/privacy/title/flyer copy is AI-drafted): a small top-up review before a wide public launch.
  - **B7 (deferred, not a v1 blocker):** engine backend customer-facing l10n (report `customer_*` +
    rejection reasons) ? cross-repo Python (`badminton-highlight-indexer`), owner-gated **D-BACKEND-DEPTH**.
- **Marketing video on the site** ? embed a short walkthrough of the record ? index ? reel process,
  themed for **serious enthusiasts**, on the landing page. ? **A real-UI CUJ screencast now exists**
```

(Playwright recording of the per-rally SPA driven against real GPU-run data ? `promo_cuj.mp4`/`.gif` in `gs://khelsutra-rally-corpus/promo/`). Decide: use it as the marketing clip (? shows real court footage + a dev "P5" badge ? clean up before public embed), and/or produce a polished Grok/Gemini cut.
- **Interactive per-rally selection** (big product feature) ? ? **MVP / M0 COMPLETE** (the full pick ? query ? select ? compile ? download loop on existing engine endpoints). Lives in the alpha SPA (`web/`). **? Tracked design + P-tracker (source of truth): `docs/PER_RALLY_SELECTION.md` (IN PROGRESS; owner-approved, D8?D11 locked).** **Done: P0 design (#11), P1 scaffold (#14), P2 grid+hook (#21), P3 `parseQuery()` query bar (#28), P4 SelectionFooter + `min_shot_count` warning (#30), P5 real load + Build?compile?preview?download (#31).** Verified e2e vs a stub engine + a CUJ replay against the real L4-run rallies + reel. **NEXT (owner-gated, engine repo): M1?M5** ? `GET /api/health` + `download_url` (P6), RESTful `?/rallies` alias (P7), multi-video home + auth (P8), per-rally preview/timeline (P9), corpus correction loop (P10), localized NL (M5). **Optional:** a fresh live `/api/process` GPU inference once the engine session's M2 training frees the single GPU.

Queued (site polish)
- **Always-Use-HTTPS + HSTS** (Cloudflare SSL/TLS) ? closes the plain-HTTP flag.
- **Cloudflare Web Analytics (owner-gated; FREE)** ? code side DONE (the dead placeholder beacon was removed from `site/public/index.html`; analytics is now via **Automatic Setup**, no in-source token). **Owner step (needs CF login):** the site is a Cloudflare **Worker** (not Pages) on the proxied `khelsutra.guru` zone ? enable via **Analytics & Logs ? Web Analytics ? Add a site ? `khelsutra.guru`** (Automatic Setup is default-on for proxied hostnames; edge-injects the beacon on every page). Then filter the view to `hostname = khelsutra.guru` to exclude `*.pages.dev`/preview noise, and decide separately whether `app.khelsutra.guru` (the SPA) needs its own enablement (history-router only). **Verify (two gotchas):** the Worker's asset responses must NOT send `Cache-Control: ?, no-transform` (Cloudflare can't rewrite that ? no injection), and after a deploy the rendered HTML should contain the edge-injected beacon. **DONE WHEN:** the Web Analytics dashboard shows incrementing page views for `khelsutra.guru`.
- **Droplet ? production** ? TLS + a subdomain (`vm.khelsutra.guru` via Caddy) if it should be a real standby; add **VM-path SMTP** so the Node host also emails (currently `noopNotify`).
- **DMARC** TXT for deliverability; delete any orphan `khelsutra-site` Worker; clean CF D1 test rows (`deploy-test@`, `email-test@`). Revoke the droplet SSH key (`~/.ssh/khelsutra_droplet`) when done.

Bigger arcs (design-first)
- **? Studio media lifecycle (ingest ? store ? annotate ? train)** ? **Tracked project: `docs/STUDIO_MEDIA_LIFECYCLE.md` (IN PROGRESS ? storage half COMPLETE 2026-07-01).** The seamless flow: add a video ? store it in a user-chosen medium (S3/GCS/Drive/local) ? free-space checked ? annotate in-UI ? contribute to the segmenter. **SHIPPED 2026-07-01:** free-space guards (vault + engine, P1/P2), the `/api/capabilities` storage block (P3),

`POST /api/assets`
71   store-into-medium (P4), the SPA medium picker (P5), the "my videos" library (P6), and in-UI annotation (P7?P9).
72   **Remaining = owner-gated only:** P10 (corpus promotion) + P11 (train/re-eval ?F1) + the `L` beta follow-ups.
73   §7 tracker is the source of truth.
74   - **Single-box appliance (KhelSutra Studio)** ? **? Tracked design: `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` (DRAFT, owner-gated).**
75   The `web/` SPA drives the full **ingest (Drive / bucket URI / local upload) ? index ? pick rallies ? reel ? download**
76   loop on ONE box, behind an **edge auth gate** (engine stays `127.0.0.1`, CORS `*` is moot). It is the single-box
77   profile of the engine's decoupled `StorageBackend` + `backend.worker` architecture. MVP runs on the CPU droplet via
78   **Gemini/stub**; **real local-GPU detection needs a GPU host** (DO droplet / GCE VM / Cloud Run+GPU ? owner priority
79   ASAP, runs parallel to the UI MVP). Per-rally selection = one screen. Owner decisions pending in §10 (edge auth,
80   retention, cost ceiling, GPU host, P5 bridge).
81   - **Alpha SPA** in `web/` (Vite + React + TS + Tailwind) ? hosts the operator console + the per-rally selection UI above.
82   - **BYO-storage + autosync** ? connect Drive/OneDrive/GCS/S3, watch the folder, auto-prepare reels
83   (engine `PLATFORM_ARCHITECTURE.md §3`). The form's GPU/storage answers are the demand signal.
84   - **Control plane** (`control-plane/`) at beta.
85
86   ## Org & repos
87   - ? **DONE (2026-06-20): proprietary repos consolidated into the `Khelsutra` org (private).** Transferred
88   `badminton-highlight-indexer` (engine), `sports-data-collector`, `sports-obsreport` ? `Khelsutra/*` (stay private; no
89   open PRs/forks; GitHub redirects git ops so active sessions kept working; local remotes repointed + verified).
90   `rally-annotator` stays in the org **PUBLIC** (OSS MIT plugin the site links to ? do not make private). Owner is org
91   admin (full access) and can **invite a contributor** (org member / per-repo outside collaborator) anytime.
92   - ? **`khelsutra` (this site repo) ? owner step pending.** A transfer breaks the **Cloudflare Workers-Builds** git
93   connection (auto-deploy) until the Cloudflare GitHub App is installed on the `Khelsutra` org and the Worker
94   reconnected (dashboard). When ready: install the CF app on the org ? `gh api repos/avidullu/khelsutra/transfer -f
95   new_owner=Khelsutra` ? reconnect the Worker to `Khelsutra/khelsutra` ? verify a deploy.
96   - ? **Engine CI under the org** ? verify the engine's GitHub Actions run on the next PR (free orgs default Actions-on;
97   enable in org settings if a run doesn't start).
98
99   ## Convention
100   Substantial work gets a tracked doc in `docs/`; one PR per item off `origin/master`; merge to `master`
101   auto-deploys (CI must be green).
102

28. user (2026-07-02T02:49:34.469Z)

deploy\digitalocean\README.md:8:> DO-specific *provisioning* (doctl droplet create, Spaces) is retired. (Optional later cleanup: move
docs\archives\decisions\DECISIONS.md:681:- The shared DO CPU droplet **`claude-do-rally-test`** (another session's testbed) is **owner-coordinated** cleanup, separate from this decision; the freshly-rotated **DO API token** can be retired once that droplet is gone.
docs\archives\past_projects\DECOUPLED_COMPUTE\DEPLOY_REPORT_GCP_2026-06-20.md:99:token was rotated; with **DO dropped for compute (ADR-013)** it can be retired once the shared DO droplet

is gone.

29. user (2026-07-02T02:49:35.951Z)

```
// DRIFT GUARD for the one build step the otherwise no-build site has (tracker B3 / D-CATALOG-HOME):
// the catalogs the marketing site ships (site/public/locales/) MUST be byte-identical to the single
// source of truth (repo-root /locales/). If someone edits /locales without re-running the copy, this
// fails CI with the exact fix. Run `npm run copy:locales` to resync.
import { test } from "node:test";
import assert from "node:assert/strict";
import { readFileSync, readdirSync } from "node:fs";
import { join } from "node:path";

const SRC = join(import.meta.dirname, "..", "..", "locales");
const PUB = join(import.meta.dirname, "..", "public", "locales");

test("site/public/locales mirrors repo-root /locales exactly (else: npm run copy:locales)", () => {
  const locales = readdirSync(SRC, { withFileTypes: true }).filter((d) => d.isDirectory()).map((d) => d.name);
  assert.ok(locales.length >= 3, "expected at least en/kn/hi catalogs in /locales");

  const shipped = new Set(readdirSync(PUB, { withFileTypes: true }).filter((d) => d.isDirectory()).map((d) => d.name));
  for (const loc of locales) {
    const src = readFileSync(join(SRC, loc, "common.json"), "utf8");
    assert.ok(shipped.has(loc), `site/public/locales/${loc} is missing ? run npm run copy:locales`);
    const pub = readFileSync(join(PUB, loc, "common.json"), "utf8");
    assert.equal(pub, src, `site/public/locales/${loc}/common.json drifted from /locales/${loc} ? run npm run copy:locales`);
  }
  // No stale extra locales shipped that aren't in the source of truth.
  for (const loc of shipped) {
    assert.ok(locales.includes(loc), `site/public/locales/${loc} has no source in /locales ? run npm run copy:locales`);
  }
});
====
locales/en:
common.json

site/public/locales:
da
en
es
hi
id
kn
```

30. user (2026-07-02T02:49:47.978Z)

```
en total=473 missing=0 extra=0
kn total=473 missing=0 extra=0
hi total=473 missing=0 extra=0
es total=473 missing=0 extra=0
da total=473 missing=0 extra=0
id total=473 missing=0 extra=0
```

31. user (2026-07-02T02:49:48.260Z)

```
// Content regression guards for the localization catalogs ? the rules the adversarial
translation
// audit (#77) established, pinned so the specific bugs can never silently return. This
complements
// hygiene.test.ts (which proves key/placeholder PARITY) by asserting catalog CONTENT. Each
block maps
// to a concrete #77 bug class; the mutation that reintroduces the bug fails here. Pure data
assertions
// over repo-root /locales (D-CATALOG-HOME) ? no render context needed.
import { describe, it, expect } from "vitest";
import { readFileSync, readdirSync } from "node:fs";
import { join } from "node:path";

const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");

type Flat = Record<string, string>;
function flatEntries(obj: Record<string, unknown>, prefix = ""): Flat {
  const out: Flat = {};
  for (const [k, v] of Object.entries(obj)) {
    const key = prefix ? `${prefix}.${k}` : k;
    if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out, flatEntries(v as
Record<string, unknown>, key));
    else out[key] = String(v);
  }
  return out;
}
const load = (loc: string): Flat =>
  flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR, loc, "common.json"), "utf8")) as
Record<string, unknown>);

const ALL_LOCALES = readdirSync(LOCALES_DIR, { withFileTypes: true })
  .filter((d) => d.isDirectory())
  .map((d) => d.name);
const en = load("en");

// (1) Internal-token / debug-tag leaks ? EVERY locale. The engine's internal `min_shot_count`
config
// name leaked into footer.floorSomeRest (#77, kn even wrapped it in <code>). No developer/debug
tag
// (kbd/samp/tt/var) or that token may reach any user-facing string. <code> is likewise banned ?
// EXCEPT on the marketing site's developer "own your model" page (site.ownModel.*), which
intentionally
// renders inline <code> (the rally_annotator.lua filename, the ending_reason enum,
<video>.rallies.csv)
// through the site shim's data-i18n-html form. That namespace is the sole, deliberate
exception; the
// SPA console keys (where the #77 leak happened) still fail on any <code>.
describe("catalog content ? no internal token / debug-tag leaks (every locale)", () => {
  const TOKEN = /min_shot_count/;
  const DEBUG_TAG = /<\/?(?:kbd|samp|tt|var)\b/i;
  const CODE_TAG = /<\/?code\b/i;
  const codeAllowed = (key: string) => key.startsWith("site.ownModel.");
  for (const loc of ALL_LOCALES) {
    it(`${loc}: no 'min_shot_count' token or stray <code>/<kbd>/<samp>/<tt>/<var> tag`, () => {
      const offenders = Object.entries(load(loc))
        .filter(([k, v]) => TOKEN.test(v) || DEBUG_TAG.test(v) || (CODE_TAG.test(v) &&
!codeAllowed(k)))
        .map(([k]) => k);
      expect(offenders).toEqual([]);
    });
  }
});
```
catalog-invariants.test.ts
hygiene.test.ts
i18n.test.tsx

```
index.ts
```

32. user (2026-07-02T02:50:07.227Z)

```
Exit code 1
site/README.md
site/make_og.py
site/package-lock.json
site/package.json
site/public/capture.html
site/public/flyer-da.html
site/public/flyer-es.html
site/public/flyer-hi.html
site/public/flyer-id.html
site/public/flyer-kn.html
site/public/flyer.html
site/public/i18n.js
site/public/img/annotate-enable.png
site/public/img/annotate-tool.png
site/public/img/flyer-qr.svg
site/public/img/studio-da.png
site/public/img/studio-en.png
site/public/img/studio-es.png
site/public/img/studio-hi.png
site/public/img/studio-id.png
site/public/img/studio-kn.png
site/public/index.html
site/public/locales/da/common.json
site/public/locales/en/common.json
site/public/locales/es/common.json
site/public/locales/hi/common.json
site/public/locales/id/common.json
site/public/locales/kn/common.json
site/public/og.png
site/public/own-model.html
====
site/access.db
check-ignore exit: 0
-rw-r--r-- 1 avidu 197609 12288 Jun 20 20:15 site/access.db
node_modules/
dist/
.env
.env.*
*.local
.DS_Store
Thumbs.db
vm-state-backups/
```

local runtime / machine-specific ? never commit
server.js creates site/access.db from schema.sql (idempotent) ? runtime only

```
*.db
```

editor/agent config: launch.json has absolute machine paths + settings.local.json + w

```
.claude/
```

33. user (2026-07-02T02:50:08.183Z)

```
==> site/scripts/droplet-smoke.sh <==
#!/usr/bin/env bash
```

Live VM smoke test: copy site/ to a droplet, boot server.js, round-trip it, report.
Proves the repo's Node + SQLite path on a REAL VM (the Cloudflare path is covered b
`npm test`). Idempotent; cleans up its temp DB. Needs Node >= 22 on the VM (or a

self-contained binary at $REMOTE_DIR/node/bin/node).

#

**SSH_TARGET=root@1.2.3.4 [SSH_KEY=~/.ssh/khelsutra_droplet] \
[REMOTE_DIR=/srv/khelsutra-site] [PORT=8787] scripts/droplet-smoke.sh**

```
set -euo pipefail
: "${SSH_TARGET:?set SSH_TARGET=user@host}"
KEY="${SSH_KEY:-$HOME/.ssh/khelsutra_droplet}"
DIR="${REMOTE_DIR:-/srv/khelsutra-site}"
PORT="${PORT:-8787}"
SITE="$(cd "$(dirname "$0")/.." && pwd)" # the site/ folder (script lives in site/scripts/)
SSHO=(-i "$KEY" -o StrictHostKeyChecking=accept-new)

echo "? scp $SITE  ?  $SSH_TARGET:$DIR/site"
ssh "${SSHO[@]}" "$SSH_TARGET" "mkdir -p '$DIR'"
scp "${SSHO[@]}" -qr "$SITE" "$SSH_TARGET:$DIR/"

ssh "${SSHO[@]}" "$SSH_TARGET" "bash -s -- '$DIR' '$PORT'" <<'REMOTE'
set -e
DIR="$1"; PORT="$2"
NODE="$( [ -x "$DIR/node/bin/node" ] && echo "$DIR/node/bin/node" || command -v node || true )"
[ -n "$NODE" ] || { echo "FAIL: no Node (>=22) on the VM"; exit 1; }
cd "$DIR/site"
DB="$DIR/smoke.db"; rm -f "$DB"
PORT="$PORT" DB_PATH="$DB" nohup "$NODE" server.js > "$DIR/smoke.log" 2>&1 &
SRV=$!
for _ in $(seq 1 30); do curl -sf -o /dev/null "http://127.0.0.1:$PORT/" && break || sleep 1;
done

==> site/scripts/provision-vm.sh <==
#!/usr/bin/env bash
```

**First-class, PROVIDER-AGNOSTIC provisioning for the KhelSutra site (Node + SQLite
Run from your LOCAL machine against a FRESH Ubuntu VM (DigitalOcean / GCE / any
it pushes site/ via `git archive` (PRIVATE-REPO-SAFE ? no clone, token, or secret on t
VM), installs Node, installs + starts a systemd service, opens the firewall, and VERIFIE
it is actually serving. Idempotent (safe to re-run). Pairs with teardown-vm.sh so a
destroy ? resume-on-a-different-provider cycle is clean and reproducible.**

#

**SSH_TARGET=root@1.2.3.4 [SSH_KEY=~/.ssh/khelsutra_droplet] \
[REMOTE_DIR=/srv/khelsutra-site] [PORT=8787] [NODE_MAJOR=24] [RESTORE_DB=
site/scripts/provision-vm.sh**

#

NODE_MAJOR must be >= 24 so `node:sqlite` is unflagged (server.js imports it directl

```
set -euo pipefail
: "${SSH_TARGET:?set SSH_TARGET=user@host}"
KEY="${SSH_KEY:-$HOME/.ssh/khelsutra_droplet}"
DIR="${REMOTE_DIR:-/srv/khelsutra-site}"
PORT="${PORT:-8787}"
NODE_MAJOR="${NODE_MAJOR:-24}"
SITE="$(cd "$(dirname "$0")/.." && pwd)"          # the site/ dir (script lives in
site/scripts/)
REPO="$(cd "$SITE/.." && pwd)"                      # repo root (for git archive)
SSHO=(-i "$KEY" -o StrictHostKeyChecking=accept-new -o ConnectTimeout=10)

echo "? [1/5] waiting for SSH at $SSH_TARGET ..."
for _ in $(seq 1 60); do ssh "${SSHO[@]}" "$SSH_TARGET" true 2>/dev/null && break || sleep 5;
done
```

```
ssh "${SSHO[@]}" "$SSH_TARGET" true                # final attempt ? fail loud if still down

echo "? [2/5] pushing site/ via git archive (private-repo-safe; no secret on the VM) ..."
ssh "${SSHO[@]}" "$SSH_TARGET" "rm -rf '$DIR/site' && mkdir -p '$DIR/site'"
```

core.autocrlf=false forces LF in the archive even when the operator runs on Windows

34. user (2026-07-02T02:50:19.269Z)

```
6f17dc0 chore(ci): bump checkout@v4?v7 and setup-node@v4?v6 (off deprecated Node 20) (#118)
5ffb750 chore(ci): retire the live-parity monitor ? droplet mirror decommissioned (#117)
commit 6f17dc024ae44ba309dba7290869cc1f11bd8b63
Author: Avi <avi.dullu@gmail.com>
Date:   Thu Jun 25 17:33:35 2026 +0530

    chore(ci): bump checkout@v4?v7 and setup-node@v4?v6 (off deprecated Node 20) (#118)

    GitHub is force-running Node-20 actions on Node 24 and warning that Node 20 is
    deprecated (2025-09-19 changelog). `test.yml` pinned `actions/checkout@v4` and
    `actions/setup-node@v4`, both Node-20 runtimes. Bump to the current majors
    (checkout v7.0.0, setup-node v6.4.0 ? both Node 24). The `node-version: "24"`
    input and `cache` settings are unchanged (setup-node@v6 keeps those inputs).

    (The other source of this warning ? the parity workflow ? was removed in #117.)

    Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

 .github/workflows/test.yml | 12 ++++++------
 1 file changed, 6 insertions(+), 6 deletions(-)
```

35. user (2026-07-02T02:50:20.939Z)

```
No matches found
```

36. user (2026-07-02T02:50:32.347Z)

```
#!/usr/bin/env bash
```

Bundle the KhelSutra web SPA into the engine so a downloaded engine serves the rea single-origin (PACKAGING_PLAN.md P1b).

```
#
```

OPTION 1 (now, owner-chosen): build the SPA from a khelsutra checkout and vendor into `backend/ui/`, which is COMMITTED ? so the wheel + CI build fully offline/reprodu the snapshot by re-running this on an SPA release; commit the diff.

```
#
```

OPTION 3 (designed-for, future): point --from at a published dist tarball/URL (a GitHub artifact) to vendor a pre-built SPA without a local khelsutra checkout or Node. Already

```
#
```

Usage:

scripts/bundle_ui.sh [SOURCE]

SOURCE = a khelsutra checkout dir (default: $KHELSUTRA_DIR or ../khelsutra)   [bu

| a path/URL ending .tgz/.tar.gz   [unpacks it]

BUILD=0 scripts/bundle_ui.sh <khelsutra-dir>   # skip npm; reuse an existing web/dis

```
set -euo pipefail

ENGINE_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
UI_DEST="$ENGINE_ROOT/backend/ui"
```

```
SOURCE="${1:-${KHELSUTRA_DIR:-$ENGINE_ROOT/../khelsutra}}"

stage_dist() {  # populates $1 (a dist dir) into a clean $UI_DEST
  local dist="$1"
  [ -f "$dist/index.html" ] || { echo "ERROR: no index.html in $dist ? not a built SPA dist"
>&2; exit 1; }
  rm -rf "$UI_DEST"
  mkdir -p "$UI_DEST"
  cp -r "$dist"/. "$UI_DEST"/
}

SPA_REPO="unknown"
SPA_COMMIT="unknown"

case "$SOURCE" in
  *.tgz|*.tar.gz|http://*|https://*)
    # OPTION 3: a published dist artifact.
    tmp="$(mktemp -d)"; trap 'rm -rf "$tmp"' EXIT
    if [[ "$SOURCE" == http*://* ]]; then
      echo "[bundle_ui] fetching dist artifact: $SOURCE"
      curl -fsSL "$SOURCE" -o "$tmp/dist.tgz"; SRC_TGZ="$tmp/dist.tgz"; SPA_REPO="$SOURCE"
```

37. user (2026-07-02T02:50:33.743Z)

```
269:    Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
408:API_VERSION = 1
424:        "api_version": API_VERSION,
450:        "api_version": API_VERSION,
```

38. user (2026-07-02T02:50:51.197Z)

```
README.md
annotate.pw.mjs
fixtures
ingest.pw.mjs
mock-engine.mjs
playwright.config.mjs
screencast.pw.mjs
store-medium.pw.mjs
to-mp4.mjs
====
====ISSUES====
114     OPEN    Cloudflare Access: an allowlisted email still couldn't sign in (alpha auth)
2026-06-23T10:25:52Z
112     OPEN    Share to WhatsApp sends only a text message, not the reel video bug
2026-06-22T18:27:52Z
111     OPEN    Resilient processing UX: survive page refresh + continually-updating progress
enhancement     2026-06-22T18:15:04Z
103     OPEN    SPA: show video stats (name, size, fps, resolution, duration) + persist them on
reload          2026-06-22T14:18:41Z
102     OPEN    SPA: &debug=1 (trace panel) / &debug=2 (full runlog) URL params for debugging +
tracing         2026-06-22T14:16:43Z
101     OPEN    SPA: show a friendly failure notice on a timeout/5xx (Cloudflare 524), not the
raw Cloudflare error HTML               2026-06-22T14:14:16Z
99      OPEN    SPA: keep the uploaded video's filename visible during processing (it disappears
mid-job)               2026-06-22T14:01:52Z
98      OPEN    SPA: replace the demo rally-selection the moment an upload/processing starts (so
the user knows their reel is coming)           2026-06-22T13:57:56Z
97      OPEN    Hosted alpha: split-origin (Pages app. + tunnel api.) breaks Cloudflare Access
for the SPA ? single-origin needed              2026-06-22T13:54:39Z
72      OPEN    i18n: add Telugu (te) locale (catalog + Noto Telugu font, key-parity green)
2026-06-21T18:17:34Z
```

39. assistant (2026-07-02T02:51:15.634Z)

No open PRs; no ESLint config in web/. Now checking the just-landed #140 code for latent issues.

40. user (2026-07-02T02:51:16.677Z)

```
// Resilient readers for a GET /api/videos row (VideoSummary). Defense-in-depth against engine drift:
// the engine SHOULD return the contract fields (`video_id`, a friendly `match_name`, `fps`,
// `resolution`), but if a build regresses to the raw DB shape (`id`, `filepath`, no name), the library
// + the persistent current-video label still work ? they read through these helpers, never the raw
// field. (The bug this guards against actually shipped once: the real engine returned `id`/no-name
// while the e2e mock faked both, so a "green" UI was broken live.)
import type { VideoSummary } from "../api/types";

/** The MD5 join key ? falls back to the raw `id` column if `video_id` is absent. */
export function videoId(v: VideoSummary): string {
  return v.video_id ?? v.id ?? "";
}

/** Filename stem of a path/URI: `?/My Match 2026.mp4` ? `My Match 2026`. Handles `/` and `\`,
 *  a trailing slash, and a no-extension name; returns "" for empty input. */
export function basenameStem(path: string | null | undefined): string {
  const base =
    String(path ?? "")
      .replace(/[/\\]+$/, "")
      .split(/[/\\]/)
      .pop() ?? "";
  const dot = base.lastIndexOf(".");
  return dot > 0 ? base.slice(0, dot) : base;
}

/** A human name ? the engine's `match_name`, else derived from the stored path/URI, else "" (the
 *  caller applies its own localized "unnamed"/"your video" fallback). */
export function videoName(v: VideoSummary): string {
  return v.match_name || basenameStem(v.local_path ?? v.filepath);
}

/** A compact spec line, e.g. `1920x1080 · 30fps` (either part optional; "" when neither is known). */
export function videoSpec(v: VideoSummary): string {
  const parts: string[] = [];
  if (v.resolution) parts.push(v.resolution);
  if (typeof v.fps === "number" && v.fps > 0) parts.push(`${Math.round(v.fps)}fps`);
  return parts.join(" · ");
}

```

41. user (2026-07-02T02:51:17.140Z)

```
// "Your videos" ? the indexed-video library (STUDIO_MEDIA_LIFECYCLE.md P6). Consumes the engine's
// GET /api/videos so a match already added can be REOPENED without re-processing (reload-survival),
// and an ingest?annotate loop across many videos has a home. Honest + quiet: on loading / error /
// empty it stays out of the way (the ingest panel below is the primary "add a new one" path).
import { useEffect, useState } from "react";
```

```
import { useTranslation } from "react-i18next";
import { listVideos } from "../api/client";
import type { VideoSummary } from "../api/types";
import { log, newCorrelationId } from "../lib/log";
import { videoId, videoName, videoSpec } from "../lib/video";

type State =
  | { status: "loading" }
  | { status: "loaded"; videos: VideoSummary[] }
  | { status: "error" };

interface Props {
  /** Called when the user reopens a stored video ? with its id and human name (for the
   *  persistent current-video label, so App needn't re-resolve it). */
  onSelect: (videoId: string, name: string) => void;
}

export function VideoLibrary({ onSelect }: Props) {
  const { t } = useTranslation();
  const [state, setState] = useState<State>({ status: "loading" });

  useEffect(() => {
    let alive = true;
    const cid = newCorrelationId();
    listVideos(cid)
      .then((videos) => alive && setState({ status: "loaded", videos }))
      .catch((err) => {
        if (!alive) return;
        // Quiet, non-blocking: the library is a convenience ? a load failure must not hide the
        // ingest panel. Log it (observability) but degrade to a soft message.
        setState({ status: "error" });
        log("warn", "library.load_failed", { cid, error: String(err) });
      });
    return () => {
      alive = false;
    };
  }, []);

  const Title = <h2 className="text-lg font-bold">{t("library.title")}</h2>;

  if (state.status === "loading") {
    return (
      <section className="mt-6" data-testid="video-library" aria-busy="true">
        {Title}
        <p className="mt-2 text-sm text-ink/50">{t("library.loading")}</p>
      </section>
    );
  }
  if (state.status === "error") {
    return (
      <section className="mt-6" data-testid="video-library">
        {Title}
        <p className="mt-2 text-sm text-ink/50">{t("library.errorLoad")}</p>
      </section>
    );
  }
  if (state.videos.length === 0) {
    return (
      <section className="mt-6" data-testid="video-library">
        {Title}
        <p className="mt-2 text-sm text-ink/50">{t("library.empty")}</p>
      </section>
    );
  }
```

```
    return (
      <section className="mt-6" data-testid="video-library">
        {Title}
        <p className="mt-1 text-sm text-ink/60">{t("library.intro")}</p>
        <ul className="mt-3 grid grid-cols-[repeat(auto-fill,minmax(220px,1fr))] gap-3 list-none p-0 m-0">
          {state.videos.map((v, i) => {
            const id = videoId(v);
            const name = videoName(v) || t("library.unnamed");
            const meta = [videoSpec(v), v.created_at].filter(Boolean).join(" · ");
            return (
              <li key={id || `row-${i}`}>
                <button
                  type="button"
                  onClick={() => onSelect(id, name)}
                  aria-label={t("library.openAria", { name })}
                  data-testid="library-item"
                  className="w-full text-left rounded-xl border border-black/10 bg-white p-3 transition hover:border-green focus:outline-none focus:ring-2 focus:ring-green"
                >
                  <div className="truncate font-semibold text-ink">{name}</div>
                  {meta && <div className="mt-1 truncate text-xs text-ink/50">{meta}</div>}
                  <span className="mt-2 inline-flex items-center text-sm font-semibold text-green">
                    {t("library.open")} ?
                  </span>
                </button>
              </li>
            );
          })}
        </ul>
      </section>
    );
  }

```

42. user (2026-07-02T02:51:35.326Z)

```
// Mock engine ? a dependency-free Node HTTP server implementing the badminton-highlight-indexer
// contract the Studio SPA calls, PLUS serving the built SPA from ./dist. One origin, so the SPA's
// default same-origin `/api/*` calls resolve here exactly as they would behind the appliance reverse
// proxy. This is the hermetic L1 layer from docs/LOCAL_APPLIANCE_ARCHITECTURE.md §8 ? deterministic,
// no GPU/key/Python ? used to record the e2e screencast and (later) drive CI CUJ replays.
//
// It is a MOCK: fixtures are canned and the job state machine advances by poll count (one "running"
// poll, then "done") so a screencast shows real progress. The response SHAPES mirror the verified
// engine contract (web/src/api/types.ts + client.ts) so the SPA drives it unmodified.
import http from "node:http";
import { readFile } from "node:fs/promises";
import { existsSync } from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const WEB_ROOT = path.resolve(__dirname, ".."); // khelsutra/web
const DIST = path.join(WEB_ROOT, "dist");
const REEL = path.join(__dirname, "fixtures", "reel.mp4");
const PORT = Number(process.env.MOCK_PORT || 8791);
```

```
const MIME = {
  ".html": "text/html; charset=utf-8", ".js": "text/javascript; charset=utf-8",
  ".mjs": "text/javascript; charset=utf-8", ".css": "text/css; charset=utf-8",
  ".json": "application/json; charset=utf-8", ".svg": "image/svg+xml", ".png": "image/png",
  ".jpg": "image/jpeg", ".jpeg": "image/jpeg", ".ico": "image/x-icon", ".webp": "image/webp",
  ".woff": "font/woff", ".woff2": "font/woff2", ".ttf": "font/ttf", ".mp4": "video/mp4",
  ".webm": "video/webm", ".map": "application/json",
};

// Deterministic rally fixtures ? the verified play_segments shape (honest fields only). id 4
sits
// below the mock's min_shot_count (5) on purpose, so the SPA's honest floor-warning is
captured.
const RALLIES = [
  { id: 1, start_time: 12,  end_time: 20,  duration: 8,  server: null, receiver: null, metadata:
{ shot_count: 9,  shot_count_confidence: "High" } },
  { id: 2, start_time: 41,  end_time: 47,  duration: 6,  server: null, receiver: null, metadata:
{ shot_count: 6 } },
  { id: 3, start_time: 63,  end_time: 84,  duration: 21, server: null, receiver: null, metadata:
{ shot_count: 19, shot_count_confidence: "Medium" } },
  { id: 4, start_time: 95,  end_time: 101, duration: 6,  server: null, receiver: null, metadata:
{ shot_count: 4 } },
  { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null, metadata:
{ shot_count: 11 } },
  { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null, metadata:
{ shot_count: 24, shot_count_confidence: "High" } },
];

const CAPS = {
  api_version: 1,
  engine_version: "mock-0.1.0",
  segmenters: ["gemini", "motion", "wasb_hybrid"],
  default_segmenter: "gemini",
  ffmpeg: { ffmpeg: true, ffprobe: true, ffmpeg_path: "/usr/bin/ffmpeg", ffprobe_path:
"/usr/bin/ffprobe" },
  gpu: { nvidia_smi_present: false },
  gemini_key_present: true,
  wasb_weights_present: false,
  deployment: { profile: "local-appliance", compute_target: "local", cloud_available: false },
  // Forward-compatible storage block (STUDIO_MEDIA_LIFECYCLE.md P3/P5). The current SPA ignores
  // unknown fields; the medium picker will render from `storage.backends` once P3/P5 land.
  storage: {
    default_medium: "local",
    backends: [
      { id: "local",  kind: "local",  label: "This computer",                 usable: true,
free_gb: 812 },
      { id: "gcs",    kind: "gcs",    label: "Google Cloud Storage bucket",   usable: true,
free_gb: null },
      { id: "s3",     kind: "s3",     label: "Amazon S3 / compatible bucket", usable: true,
free_gb: null },
      { id: "gdrive", kind: "gdrive", label: "Google Drive folder (mounted)", usable: true,
free_gb: 1900 },
```
====

Vendored: rally-annotator annotation core (MIT)

`csv.ts` and `annotator.ts` here are **vendored verbatim** from the open-source, MIT-licensed **[rally-annotator](https://github.com/Khelsutra/rally-annotator)** (`web/src/state/`), synced at commit **`3d604bc`**. They are the pure, framework-free labeling core ? the rally **CSV** (`RallyRow` + serialize/parse, byte-compatible with the VLC plugin and the browser extension) and the **`Annotator` state machine** (mark start/end · reason · shots · save/edit/delete/undo, with an injectable `PersistFn`). No `chrome`/`wxt`/DOM coupling.

Why vendored (not an npm dependency)

Studio embeds the labeler as an in-browser React panel (`STUDIO_MEDIA_LIFECYCLE.md` D10/D13).
Vendoring
the *pure core* ? rather than publishing a library from the public repo and depending on it ?
keeps the
public MIT repo **untouched** (no Studio glue leaks in), avoids cross-repo release coupling, and
lets
Studio build a native React UI + inject its own video handler and persist (POST to the engine).
MIT
permits this use in a proprietary product provided the copyright + license notice travel with
the copy
(below).

Rules

- **Do not edit these two files here.** Fix upstream in rally-annotator, then re-sync (update
the commit
  above). Keeping them byte-compatible is what lets the labels ingest into the same pipeline
  (`sports-data-collector` ? `badminton-highlight-indexer`), joined by the video's file MD5.
- The Studio-specific glue (video handler, POST-to-engine persist, save-state UI, Studio
strings) lives
  in `web/src/components/AnnotatePanel.tsx`, **not** here.

License (MIT ? from rally-annotator/LICENSE)

43. user (2026-07-02T02:51:45.392Z)

```
5ec1831 feat(web): i18n infrastructure + English catalog (keys-not-prose) (#18)
ad6ac9c Add rally-annotator web extension (Chrome MV3) with proof-driven tests + CI (#14)
3d604bc docs: sync READMEs with current state (multi-agent audit fixes) (#25)
3d604bc is ancestor
```

44. user (2026-07-02T02:51:46.735Z)

```
site/wrangler.toml:1:# Cloudflare Worker for khelsutra.guru ? static site + Request-Access form
API.
site/wrangler.toml:8:name = "hidden-king-khelsutra"  # the git-connected Worker that owns
khelsutra.guru ? keep in sync with Cloudflare
site/wrangler.toml:27:NOTIFY_FROM = "noreply@khelsutra.guru"
docs/DRY_RUN_CUJ.md:3:> **Status:** `DONE ? DELIVERED (2026-06-21)` · **Owner:** Avi Dullu ·
**Created:** 2026-06-21. DoD §6 boxes 1?3 ? (Path A + Path B/B-P2 live e2e); the only open box
```

(droplet, §6 box 4) is **owner-parked** (GPU subscription). Archive once that box closes.
docs/DRY_RUN_CUJ.md:30:1. **Engine host + deps:** run the engine FastAPI (`python -m backend.main --server`, binds `127.0.0.1:8000`) on a box with `ffmpeg` + OpenCV. Gemini/motion paths are **CPU-only, no WSL** ? local Windows works; the CPU droplet works too. (GPU/WSL only needed for native WASB.)
docs/DRY_RUN_CUJ.md:42:3. **Engine host:** local Windows (simplest, Gemini/motion) · the CPU droplet `168.144.126.176` (shared testbed ? careful) · a GPU VM (only for WASB).
docs/DRY_RUN_CUJ.md:56:- [ ] *(option 3)* the same CUJ confirmed on the **CPU droplet** (Linux deploy target) ? droplet reachable but currently **un-provisioned** (no engine checkout; ~3.8GB RAM) ? needs a from-scratch engine install or a proper deploy of the fixed engine.
docs/DRY_RUN_CUJ.md:69:The async-job server **hung every job in `queued`** when run the documented way (`python -m backend.main`): main.py runs as `__main__` (sets `__main__.db_instance`), but `job_worker` reads `import backend.main` ? a **separate** module whose `db_instance` is `None` ? the drain crashes silently. Latent because the unit tests monkeypatch the executor inline and the UI was verified vs a stub engine. **Fix + a real-subprocess regression test ? engine [#281](https://github.com/Khelsutra/badminton-highlight-indexer/pull/281)** (dispatch through `backend.main`; FAILS-without / PASSES-with). Unblocked the run via the single-module invocation; the fix applies to the droplet too.

45. assistant (2026-07-02T02:52:04.494Z)

Let me disambiguate the rally-annotator vendor-sync state.

46. user (2026-07-02T02:52:07.250Z)

HEAD:
3d604bc docs: sync READMEs with current state (multi-agent audit fixes) (#25)
state-touching commits after 3d604bc:
diff stat:
(empty = in sync)

47. user (2026-07-02T02:52:21.909Z)

```
    @app.post("/api/assets", status_code=201)
    def create_asset(req: AssetRequest, request: Request):
        """Store a video into the user's chosen medium and register it as a library asset (P4, D14).

        SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root`` is set ? the
        hosted alpha jails inputs and has no per-user medium credentials, so it stays upload-to-local.
        Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a local file from
        the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An uploaded file +
        a REMOTE medium ? a durable, MD5-keyed copy is ``put`` into the medium while the local upload
        stays as the working copy; a ``local`` medium keeps the upload in place (it's already on this box);
        a pasted at-rest URI is registered as-is (no copy, D14b). The asset is keyed by the whole-file
        MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears in
        ``GET /api/videos``."""
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        cfg = request.app.state.config
        sec = getattr(cfg, "security", None)
        if getattr(sec, "allowed_video_root", None):
            raise HTTPException(
```

```
                status_code=403,
                detail=(
                    "storing into a custom medium is disabled on this hosted deployment "
                    "(security.allowed_video_root is set); upload to local instead."
                ),
            )
        if bool(req.uploaded_path) == bool(req.source_uri):
            raise HTTPException(
                status_code=400,
                detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
            )
        if req.uploaded_path and not req.medium_uri:
            raise HTTPException(
                status_code=400, detail="'medium_uri' is required when uploading a file."
            )

        storage_settings = _storage_settings(cfg)

        # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key, D3).
        if req.uploaded_path:
            try:
                validate_input_path(req.uploaded_path, allowed_root=None)
                src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
                raise HTTPException(
                    status_code=404,
                    detail=f"uploaded file not found at '{req.uploaded_path}'",
                )
            local_working = src_ref.key
        else:
            try:
                validate_input_path(req.source_uri, allowed_root=None)
                src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that scratch (P2).
            if src_ref.backend != "local":
                need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
                require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
            try:
                local_working = storage.acquire_local_input(
                    req.source_uri, getattr(cfg, "storage", None)
                )
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            except Exception as e:  # noqa: BLE001 ? a fetch miss is a 502, surfaced not swallowed
                raise HTTPException(
                    status_code=502, detail=f"could not resolve source_uri: {e}"
                ) from e
            if not os.path.isfile(local_working):
                raise HTTPException(
                    status_code=404, detail="source_uri did not resolve to a readable file."
                )

        video_id = compute_video_id(local_working)
        ext = os.path.splitext(local_working)[1] or ".mp4"

        # 2) Decide the durable location + copy only when needed.
        copied = False
        if req.source_uri:
```

```
811             stored_uri = req.source_uri  # already at rest in its medium ? no copy (D14b)
```

48. user (2026-07-02T02:52:23.644Z)

{"state":"OPEN","title":"refactor(D1/P10): extract orchestration.py from main.py (god-module split, step 1)","updatedAt":"2026-06-26T14:37:18Z"}

49. user (2026-07-02T02:52:36.395Z)

No matches found

50. user (2026-07-02T02:52:37.923Z)

```
web/src/api/client.ts:87: *  processing. POST /api/process {video_path} ? 202 {job_id, ...}
[main.py:260]. Fast-fails 400
web/src/api/client.ts:105:/** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ?
unknown job_id. */
web/src/api/client.ts:110:/** List indexed videos, newest first ? the "my videos" library. GET
/api/videos [main.py:447].
web/src/api/client.ts:156:/** List a video's detected rallies. POST /api/query {video_id} ?
{play_segments} [main.py:331]. */
web/src/api/client.ts:223:/** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
web/src/api/client.ts:228:/** Read engine config (for the stitching floor). GET /api/config
[main.py:121]. */
web/src/api/client.ts:238:/** The shot-count floor /api/compile silently enforces ? the UI must
warn, not hide [main.py:362]. */
web/src/api/client.ts:247:/** Start a chunked upload. 400 = bad extension; 413 = over the size
limit [uploads.py:64]. */
web/src/api/client.ts:256:/** Append ONE part (raw bytes ? not JSON). Parts are strictly
sequential 0,1,2,? [uploads.py:89]. */
web/src/api/client.ts:288:/** Finalize: returns {path, video_id} ? feed `path` to processVideo()
[uploads.py:123]. */
web/src/api/client.ts:297:/** Abort an in-flight upload (best-effort cleanup of the server-side
partial) [uploads.py:147]. */
web/src/api/types.ts:3:// backend/main.py:331-396 (query/compile), :307-328 (highlights), :121
(config).
web/src/api/types.ts:33:/** One row of the indexed-video library. GET /api/videos
[backend/main.py:447] returns the most
web/src/api/types.ts:143:/** POST /api/process ? 202 [backend/main.py:260]. The job is queued;
poll GET /api/jobs/{job_id}. */
web/src/api/types.ts:172:/** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
```

51. user (2026-07-02T02:53:12.359Z)

```
tracked-count: 0
web/dist
ignored
```

52. user (2026-07-02T02:53:13.178Z)

```
web/src/api/client.test.ts:230:    vi.stubGlobal("fetch", vi.fn(async () => { throw new
Error("offline"); }) as unknown as FetchFn);
web/src/components/a8.status.test.tsx:102:      throw new Error("offline");
web/src/components/build-error.test.tsx:58:      throw new Error("offline");
web/src/components/build-error.test.tsx:92:      if (u === "/api/query" && method === "POST")
throw new Error("offline"); // engine down on load
web/src/components/FfmpegBanner.test.tsx:64:        throw new Error("offline");
web/src/components/FfmpegBanner.tsx:9:// engine doesn't report the ffmpeg block at all, it stays
hidden ? the offline banner / wizard cover
web/src/components/IngestPanel.test.tsx:114:      throw new Error("offline");
web/src/components/QueryBar.test.tsx:4:// live parsed-pill, and the "unreadable" branch were
untested (funcs 40%). The offline parser
```

53. user (2026-07-02T02:53:25.338Z)

```
0
     14 annotate
     10 medium
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329)
(#339)
293e5c0 feat(pkg): re-bundle backend/ui ? serve the P2c first-run wizard from indexer --app
(#312)
4a06b8b feat(pkg): P1b ? bundle the KhelSutra SPA single-origin into the engine wheel (default-
OFF) (#302)
```

54. assistant (2026-07-02T02:54:14.194Z)

Let me quickly check CI status on khelsutra master and the issue-overlap evidence for #99/#103.

55. user (2026-07-02T02:54:23.942Z)

```
completed       success web: resilient video-summary readers + show fps in the library (#140)
test    master  push    28532935786     1m28s   2026-07-01T16:38:03Z
completed       success docs: refresh khelsutra docs for today's storage-track work (P2-P5) (?
test    master  push    28530208455     1m29s   2026-07-01T15:51:14Z
completed       success web: persistent "FFmpeg not detected" banner (hard-blocker warning) (?
test    master  push    28530204391     1m14s   2026-07-01T15:51:10Z
====
OPEN SPA: keep the uploaded video's filename visible during processing (it disappears mid-job)
OPEN SPA: show video stats (name, size, fps, resolution, duration) + persist them on reload
```

56. user (2026-07-02T02:55:34.880Z)

Structured output provided successfully